Lumio — Data Analytics Capstone Project

See what your systems are hiding: detecting silent payment failures with statistics, regression, time series, machine learning and Monte Carlo simulation

Project type	End-to-end data analytics project (CRISP-DM)
Domain	Digital payments operations (banking / fintech)
Data	31 days of payment telemetry: 3-week training history (17 Jan – 6 Feb 2025) + 10-day live period (7 – 16 Feb 2025), 48 segments, 5-minute grain, ~430,000 rows. Rhythm and three real anomalies from the **Numenta Anomaly Benchmark (NAB)**
Tools	Python · pandas · SQL (SQLite) · SciPy · statsmodels · scikit-learn · matplotlib / seaborn
Companion app	The Lumio live demo (`../run_local.py`), which uses the same dataset and runs these methods in real time

How to run: `pip install -r requirements.txt` , then open this notebook in Jupyter or VS Code and choose *Run All* (about 2–3 minutes). Every number in the text below is computed by the code; nothing is typed in by hand.

Contents

#	Section	Analytics skill demonstrated
0	Project framing (CRISP-DM)	Business understanding, objectives, success criteria, hypotheses
1	Data understanding	Data sourcing, data dictionary
2	Data quality & preparation	Missing data, duplicates, outliers, aggregation
3	Exploratory data analysis	Descriptive statistics, distributions, seasonality, correlation
4	SQL analysis	GROUP BY, CASE, JOIN, CTE, window functions (RANK, LAG, rolling AVG)
5	Hypothesis testing	t-test, chi-square, ANOVA + Tukey, Kruskal-Wallis, Mann-Whitney, Fisher, confidence intervals, effect sizes
6	Linear regression	Simple & multiple OLS, confounding, interpretation, assumptions & diagnostics, train/test evaluation
7	Time-series analysis	Decomposition, stationarity (ADF/KPSS), ACF/PACF, AIC model selection, seasonal naive, Holt-Winters, SARIMA, walk-forward validation, prediction intervals
8	Classification & anomaly detection	Logistic regression, odds ratios, confusion matrix, ROC/AUC, precision-recall, threshold trade-off, event-level evaluation
9	Machine learning & explainability	Gradient boosting vs linear, time-series cross-validation, permutation importance, partial dependence
10	Clustering & segmentation	K-means, elbow, silhouette, PCA
11	Monte Carlo simulation	Business-case risk simulation, convergence, sensitivity (tornado), alert-design simulation
12	Ethics, privacy & governance	Responsible analytics
13	Results, limitations & recommendations	Communicating findings
14	Professor's likely questions	Viva / defence preparation

---

0. Project framing (CRISP-DM)

This project follows **CRISP-DM** (Cross-Industry Standard Process for Data Mining), the most widely used framework for analytics projects:

CRISP-DM phase	Where it happens in this project
1. Business understanding	Section 0: problem, stakeholders, questions, success criteria
2. Data understanding	Sections 1, 3: sources, dictionary, EDA
3. Data preparation	Section 2: quality checks, cleaning, aggregation, feature engineering
4. Modelling	Sections 5–11: statistics, regression, time series, classification, ML, clustering, simulation
5. Evaluation	Sections 6.4, 7.5, 8, 9, 11: accuracy, precision/recall, cross-validation, business value
6. Deployment	The Lumio live app: the same models running on streaming data, with alerts e-mailed to teams

0.1 Business problem

Banks lose money and customers to **"silent" payment failures**: problems that never trip a technical alarm because every server is "up", yet payments slow down, fail, or quietly vanish for one slice of customers (one bank, one app version, one region). Fixed-threshold monitoring either misses them or floods teams with false alarms, especially when normal demand swings by 10× between 4 am and 6 pm, on weekends, on payday, or during a storm.

0.2 Stakeholders

Stakeholder	What they need
Operations / SRE	Fewer, more accurate alerts, with a probable cause
Product & Release teams	To know quickly when a new app version hurts payments
Fraud & Risk	Early sign of attacks (e.g. card-testing bots)
Business Operations	To tell demand changes (storm, payday) apart from faults
Leadership	Money at risk and the return on investment (ROI) of the tool

0.3 Analytical questions → hypotheses

#	Question	Hypothesis tested	Section
Q1	What does "normal" look like?	Volume and payment time follow daily and weekly seasonality	3, 7
Q2	Do payment channels, platforms and banks behave differently?	H_0: mean payment time is equal across groups	5
Q3	What drives payment time?	Payment time depends on channel, bank, hour, load and upstream services	6, 9
Q4	Can we forecast demand?	A seasonal model beats a naive forecast	7
Q5	Can we detect real incidents without flooding teams?	A model on *expected vs actual* beats a fixed threshold on precision **and** recall	8
Q6	Are there natural groups of segments?	Segments cluster by behaviour	10
Q7	Is it worth it?	Faster detection reduces annual losses by more than the tool costs	11

0.4 Success criteria (set before modelling)

- **Statistical:** regression R^2 on unseen data ≥ 0.5; forecast MAPE ≤ 10 % on normal days.
- **Detection:** catch every planted incident in the test period with fewer false alarms than a fixed-threshold rule.

- **Business:** positive expected net benefit in the Monte Carlo simulation, with a stated probability.

0.5 Scope & assumptions

- The unit of analysis is a **segment × 5-minute bucket** (aggregated telemetry, **no customer-level or personal data**).
- The **answer key** (planted events) is used *only* for evaluation, never for training the detection rules, just as in the live app.
- Monetary assumptions in Section 11 are clearly labelled and tested with sensitivity analysis.

0.6 Project plan & deliverables

Week	Phase	Activities	Deliverable
1	Business understanding	Stakeholder interviews, problem statement, success criteria	Project charter (Section 0)
2	Data understanding	Source NAB data, design the simulation, data dictionary	Dataset + dictionary (Section 1)
3	Data preparation	Quality checks, cleaning rules, aggregation, features	Clean analytical tables (Section 2)
4	Exploration	EDA, SQL analysis, hypothesis tests	EDA report (Sections 3–5)
5–6	Modelling	Regression, time series, classification, ML, clustering	Models + diagnostics (Sections 6–10)
7	Evaluation & business case	Accuracy metrics, detector comparison, Monte Carlo ROI	Evaluation + business case (Sections 8, 11)
8	Deployment & communication	Live dashboard, email alerts, report, presentation	Lumio live app, this notebook, slide deck

0.7 Data analytics skills map

Skill area	Techniques used	Section
Data wrangling	Deduplication, gap handling, weighted aggregation, feature engineering	2
Descriptive statistics	Mean, median, percentiles, skewness, kurtosis, IQR	2, 3
Visualisation	Histograms, heatmaps, box plots, time-series charts, ROC/PR curves	throughout
SQL	Aggregation, CASE, JOIN, CTE, RANK, LAG, rolling windows	4
Inferential statistics	t-test, χ^2, ANOVA, Tukey, Kruskal-Wallis, Mann-Whitney, Fisher, confidence intervals, effect sizes	5
Regression	Simple & multiple OLS, log transform, confounding, diagnostics (BP, DW, VIF), robust SE	6
Time series	MSTL, ADF/KPSS, ACF/PACF, Holt-Winters, SARIMA, AIC vs validation, walk-forward	7
Classification	Logistic regression, odds ratios, imbalance, confusion matrix, ROC-AUC, PR-AUC	8
Machine learning	Gradient boosting, time-series cross-validation, permutation importance, partial dependence	9
Unsupervised learning	Standardisation, K-means, elbow, silhouette, PCA	10
Simulation & decision analysis	Monte Carlo, bootstrap, difference-in-differences, convergence, tornado sensitivity, break-even	11
Responsible analytics	Privacy, fairness, explainability, human-in-the-loop	12

Setup

```
import warnings; warnings.filterwarnings("ignore")
from pathlib import Path
import json, sqlite3, time
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, matplotlib.dates as mdates, seaborn as sns
from IPython.display import display, Markdown
from scipy import stats
import statsmodels.api as sm, statsmodels.formula.api as smf
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.proportion import proportion_confint
from statsmodels.tsa.seasonal import MSTL
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (confusion_matrix, classification_report, roc_curve,
roc_auc_score,
    precision_recall_curve, average_precision_score, precision_score,
recall_score, f1_score,
    mean_absolute_error, mean_squared_error, r2_score, silhouette_score,
ConfusionMatrixDisplay)
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import TimeSeriesSplit, cross_validate
from sklearn.inspection import permutation_importance, PartialDependenceDisplay

DATA = next(p for p in [Path("../data/dataset"), Path("data/dataset"),
Path("lumio-live-demo/data/dataset")] if p.exists())
SEED = 42
rng = np.random.default_rng(SEED)
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams.update({"figure.figsize": (11, 4), "figure.dpi": 110,
"axes.titleweight": "bold",
                     "axes.titlesize": 12, "axes.spines.top": False,
"axes.spines.right": False})
TEAL, AMBER, INK, RED, GREY, BLUE = "#0F766E", "#D97706", "#14161A", "#B91C1C",
"#9CA3AF", "#2563EB"
pd.set_option("display.max_columns", 40); pd.set_option("display.width", 200)
RESULTS = {}          # key numbers collected for the summary in Section 13
def say(text): display(Markdown(text))
print("Data folder:", DATA)
```

```
Data folder: ../data/dataset
```

---

1. Data understanding

1.1 Sources

File	Rows	What it is
`payments_history_3weeks.csv.gz`	~290 k	Training history: 17 Jan – 6 Feb 2025, normal operations
`payments_demo_10days.csv.gz`	~139 k	Live period: 7 – 16 Feb 2025 (6 business days + 2 weekends), contains the anomalies
`services_*.csv.gz`	~60 k	Upstream services (UPI switch, card network, fraud check, 4 core-banking systems)
`answer_key.csv`	19	The planted events: **used only to evaluate**
`feeds.csv`	6	Deploy feed and change calendar (context a real analyst would also have)

Provenance. Volume follows the real NYC-taxi demand trace from the **Numenta Anomaly Benchmark** (MIT licence), including the real January 2015 blizzard. Two services follow real NAB server traces (an EC2 system failure and an RDS CPU level shift). The payments themselves are simulated on top of these real rhythms, so the ground truth is known exactly. This is what makes rigorous evaluation possible.

```
hist_raw = pd.read_csv(DATA / "payments_history_3weeks.csv.gz", parse_dates=
["ts"])
demo_raw = pd.read_csv(DATA / "payments_demo_10days.csv.gz",   parse_dates=["ts"])
svc = pd.concat([pd.read_csv(DATA / "services_history_3weeks.csv.gz", parse_dates=
["ts"]),
                 pd.read_csv(DATA / "services_demo_10days.csv.gz",   parse_dates=
["ts"])], ignore_index=True)
key   = pd.read_csv(DATA / "answer_key.csv", parse_dates=["start", "end"])
key["scope"] = key["scope"].apply(json.loads)
feeds = pd.read_csv(DATA / "feeds.csv", parse_dates=["ts"])
SEG = ["bank", "channel", "platform", "region"]

print(f"History : {len(hist_raw):>8,} rows  {hist_raw.ts.min()} → 
{hist_raw.ts.max()}")
print(f"Live    : {len(demo_raw):>8,} rows  {demo_raw.ts.min()} → 
{demo_raw.ts.max()}")
print(f"Services: {len(svc):>8,} rows  ({svc.service.nunique()} services)")
print(f"Segments: {hist_raw.groupby(SEG).ngroups} = " + " × ".join(f"
{hist_raw[c].nunique()} {c}s" for c in SEG))
display(demo_raw.head())
```

```
History :  290,304 rows  2025-01-17 00:00:00 → 2025-02-06 23:55:00
Live    :  138,558 rows  2025-02-07 00:00:00 → 2025-02-16 23:55:00
Services:   62,496 rows  (7 services)
Segments: 48 = 4 banks × 2 channels × 2 platforms × 3 regions
```

	ts	bank	channel	platform	region	app_version	volume	success_count	p95_ms	avg_amount
0	2025-02-07	Bank X	UPI	Android	North	5.1	409	392	1774.8	37.54
1	2025-02-07	Bank X	UPI	Android	South	5.1	340	334	1818.9	37.69
2	2025-02-07	Bank X	UPI	Android	West	5.1	237	236	1964.2	34.51
3	2025-02-07	Bank X	UPI	iOS	North	5.1	236	232	1732.0	38.44
4	2025-02-07	Bank X	UPI	iOS	South	5.1	213	209	1699.6	41.24

1.2 Data dictionary

Column	Type	Meaning	Example
`ts`	datetime	Start of the 5-minute bucket	2025-02-07 10:20
`bank`	category	Issuing bank (anonymised)	Bank X

Column	Type	Meaning	Example
`channel`	category	Payment rail	UPI / Card
`platform`	category	Customer app platform	Android / iOS
`region`	category	Region	North / South / West
`app_version`	category	App version in use	5.1
`volume`	count	Payments attempted in the bucket	409
`success_count`	count	Payments that succeeded	392
`p95_ms`	ms	95th-percentile payment time (95 % of payments were faster)	1774.8
`avg_amount`	$	Average payment value	37.54
Derived			
`failed`	count	`volume − success_count`	17
`success_rate`	ratio	`success_count / volume`	0.958
`hour` , `dow` , `is_weekend`	int	Calendar features	10, 4, 0
`segment`	text	Bank · Channel · Platform · Region	Bank X · UPI · Android · North

Why p95 and not the average? Payment times are right-skewed: a few very slow payments ruin the customer experience but barely move the mean. The 95th percentile is the industry-standard service-level indicator (SLI).

---

2. Data quality & preparation

Good analysis starts with trustworthy data. We check **completeness, uniqueness, validity and consistency**, the standard data-quality dimensions.

```
def dq_profile(df, name):
    k = ["ts"] + SEG + ["app_version"]
    seg_per_ts = df.drop_duplicates(["ts"] + SEG).groupby("ts").size()
    return pd.Series({
        "rows": len(df),
        "missing values (nulls)": int(df.isna().sum().sum()),
        "duplicate records": int(df.duplicated(k).sum()),
        "timestamps": df.ts.nunique(),
        "timestamps with < 48 segments reporting": int((seg_per_ts < 48).sum()),
        "invalid: success > volume": int((df.success_count > df.volume).sum()),
        "invalid: volume ≤ 0": int((df.volume <= 0).sum()),
        "invalid: p95 ≤ 0": int((df.p95_ms <= 0).sum()),
    }, name=name)
dq = pd.concat([dq_profile(hist_raw, "history"), dq_profile(demo_raw, "live")],
axis=1)
display(dq)
```

	history	live
rows	290304	138558
missing values (nulls)	0	0
duplicate records	0	16
timestamps	6048	2880
timestamps with < 48 segments reporting	0	18
invalid: success > volume	0	0
invalid: volume ≤ 0	0	0
invalid: p95 ≤ 0	0	0

```
# Where are the gaps and duplicates in the live period?
seg_per_ts = demo_raw.drop_duplicates(["ts"] + SEG).groupby("ts").size()
gap = seg_per_ts[seg_per_ts < 48]
runs = (gap.index.to_series().diff() != pd.Timedelta("5min")).cumsum()
gi = gap.index.to_series()
gap_tbl = pd.DataFrame({"from": gi.groupby(runs.values).min(),
                        "to": gi.groupby(runs.values).max() +
pd.Timedelta("5min"),
                        "segments reporting": gap.groupby(runs.values).min()})
dups = demo_raw[demo_raw.duplicated(["ts"] + SEG + ["app_version"], keep=False)]
print("Missing-data windows (a monitoring collector stopped sending):");
display(gap_tbl)
print("Duplicated records:"); display(dups.groupby(["bank", "region"]).agg(first=
("ts", "min"), last=("ts", "max"), rows=("ts", "size")))

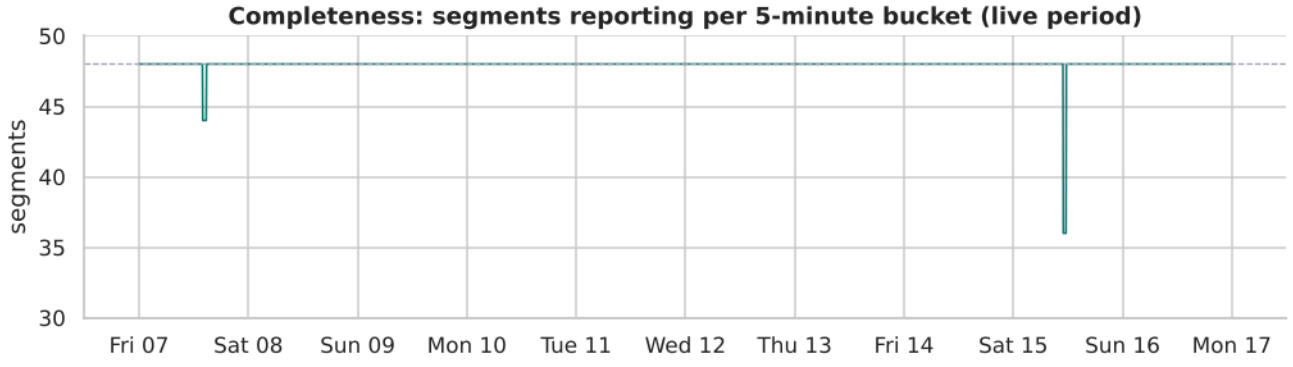
fig, ax = plt.subplots(figsize=(11, 2.6))
ax.plot(seg_per_ts.index, seg_per_ts.values, color=TEAL, lw=1)
ax.set(title="Completeness: segments reporting per 5-minute bucket (live period)",
ylabel="segments", ylim=(30, 50))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%a %d"))
ax.axhline(48, color=GREY, ls="--", lw=.8); plt.show()
```

```
Missing-data windows (a monitoring collector stopped sending):
```

	from	to	segments reporting
1	2025-02-07 14:00:00	2025-02-07 14:50:00	44
2	2025-02-15 11:00:00	2025-02-15 11:40:00	36

```
Duplicated records:
```

		first	last	rows
bank	**region**			
Bank W	**North**	2025-02-12 09:30:00	2025-02-12 09:45:00	32




```
# Outliers (IQR rule) in normal history: are they errors or real behaviour?
def iqr_outliers(s):
```

```
    q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1
    return (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)
out = hist_raw.groupby("channel")["p95_ms"].transform(iqr_outliers)
print(f"IQR outliers in p95 (history): {out.mean():.2%} of rows")
print(hist_raw[out].groupby("hour" if "hour" in hist_raw else
hist_raw.ts.dt.hour).size().sort_values(ascending=False).head(5).rename("outlier
rows by hour"))
```

```
IQR outliers in p95 (history): 1.54% of rows
ts
18    580
19    556
20    476
21    436
4     315
Name: outlier rows by hour, dtype: int64
```

Decisions (documented, as in a real project):

1. **Duplicates** → removed. Counting the same payments twice would inflate volume and could *invent* an anomaly.
2. **Missing windows** → **not** filled with zeros. A zero would look like "all payments vanished", a false alarm. They are left missing and reported to the data team. (The live app does exactly this: it holds the data and raises a *data ticket*, not a page.)
3. **IQR outliers** → **kept**. They cluster at the busy hours: they are real peak-time behaviour, not errors. Deleting them would teach the model that peaks don't exist.
4. **App-version rows** → during a rollout one segment reports two versions. For segment-level analysis we combine them with **volume-weighted** averages; for the root-cause test (5.4) we keep them separate.

```
def prepare(df, period):
    df = df.drop_duplicates(["ts"] + SEG + ["app_version"]).copy()
    df["w_p95"], df["w_amt"] = df.p95_ms * df.volume, df.avg_amount * df.volume
    g = df.groupby(["ts"] + SEG, as_index=False).agg(volume=("volume", "sum"),
success_count=("success_count", "sum"),
                                                     w_p95=("w_p95", "sum"),
w_amt=("w_amt", "sum"))
    g["p95_ms"], g["avg_amount"] = g.w_p95 / g.volume, g.w_amt / g.volume
    g = g.drop(columns=["w_p95", "w_amt"])
    g["failed"] = g.volume - g.success_count
    g["success_rate"] = g.success_count / g.volume
    g["hour"], g["dow"] = g.ts.dt.hour, g.ts.dt.dayofweek
    g["weekday"] = g.ts.dt.day_name().str[:3]
    g["is_weekend"] = (g.dow >= 5).astype(int)
    g["date"] = g.ts.dt.normalize()
    is_payday = g.date.isin(pd.to_datetime(["2025-01-31", "2025-02-14"]))   # 15th
/ month-end, moved to Friday
    g["payday_evening"] = (is_payday & g.hour.between(16, 22)).astype(int)
    g["log_p95"], g["log_volume"] = np.log(g.p95_ms), np.log(g.volume)
    g["segment"] = g.bank + " · " + g.channel + " · " + g.platform + " · " +
g.region
    g["period"] = period
    return g

hist, live = prepare(hist_raw, "history"), prepare(demo_raw, "live")
pay = pd.concat([hist, live], ignore_index=True)
for c in SEG + ["segment"]: pay[c] = pay[c].astype("category")
```

```
hist, live = pay[pay.period == "history"].copy(), pay[pay.period == "live"].copy()

# Mask of rows touched by any planted event (used ONLY for evaluation / "normal-
only" test sets)
def in_scope(df, scope):
    m = np.ones(len(df), bool)
    for k, v in scope.items(): m &= (df[k] == v).values
    return m
def event_mask(df, kinds=None, pad="0min"):
    m = np.zeros(len(df), bool)
    for _, e in key.iterrows():
        if kinds and e.kind not in kinds: continue
        m |= in_scope(df, e.scope) & ((df.ts >= e.start - pd.Timedelta(pad)) &
(df.ts < e.end + pd.Timedelta(pad))).values
    return m
live["any_event"] = event_mask(live, pad="30min")
print(f"Clean segment-level table: {len(pay):,} rows | live rows touched by a
planted event: {live.any_event.mean():.1%}")
```

```
Clean segment-level table: 428,408 rows | live rows touched by a planted event:
21.2%
```

---

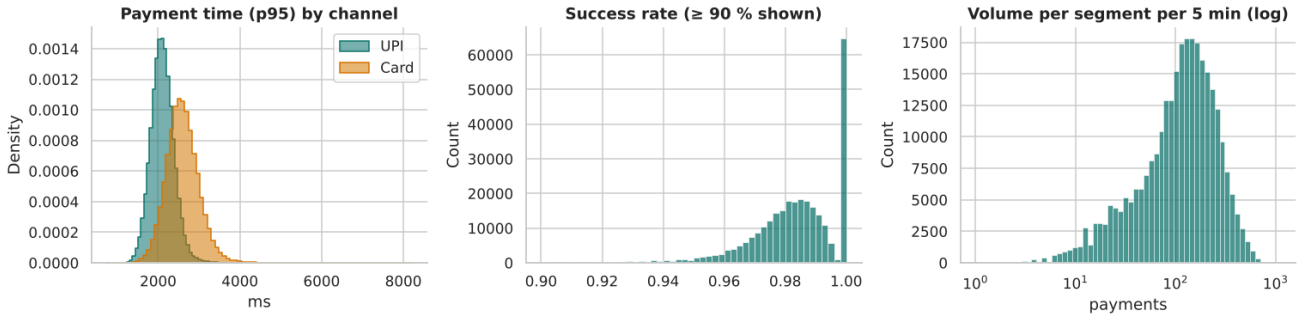
3. Exploratory data analysis (EDA)

EDA on the **history only**, so we learn what *normal* looks like without peeking at the live period.

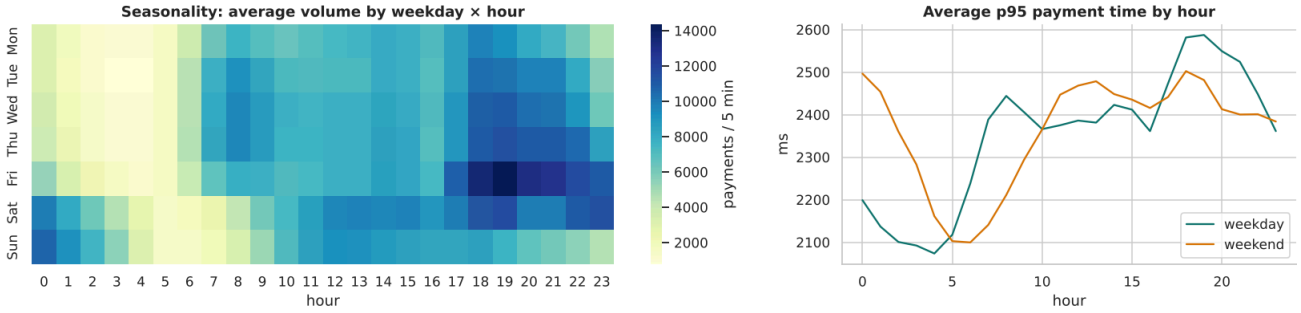
```
num = ["volume", "success_rate", "p95_ms", "avg_amount"]
desc = hist[num].describe(percentiles=[.05, .25, .5, .75, .95]).T
desc["skew"], desc["kurtosis"] = hist[num].skew(), hist[num].kurt()
display(desc.round(3))
```

	count	mean	std	min	5%	25%	50%	75%	95%	max	skew	kurt
volume	290304.0	142.276	108.615	1.00	17.000	62.000	119.000	195.000	353.00	1121.00	1.457	3
success_rate	290304.0	0.983	0.017	0.00	0.957	0.975	0.984	0.994	1.00	1.00	-3.767	73.
p95_ms	290304.0	2354.064	431.362	741.30	1741.800	2048.400	2305.900	2615.600	3105.10	8206.50	0.903	3.
avg_amount	290304.0	51.069	13.282	30.25	35.650	38.000	49.285	64.000	68.25	79.71	0.057	-1

```
fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
for ch, col in [("UPI", TEAL), ("Card", AMBER)]:
    sns.histplot(hist[hist.channel == ch].p95_ms, bins=80, ax=ax[0], color=col,
label=ch, stat="density", alpha=.55, element="step")
ax[0].set(title="Payment time (p95) by channel", xlabel="ms"); ax[0].legend()
sns.histplot(hist.success_rate[hist.success_rate >= .9], bins=50, ax=ax[1],
color=TEAL); ax[1].set(title="Success rate (≥ 90 % shown)", xlabel="")
sns.histplot(hist.volume, bins=60, ax=ax[2], color=TEAL, log_scale=True);
ax[2].set(title="Volume per segment per 5 min (log)", xlabel="payments")
plt.tight_layout(); plt.show()
```

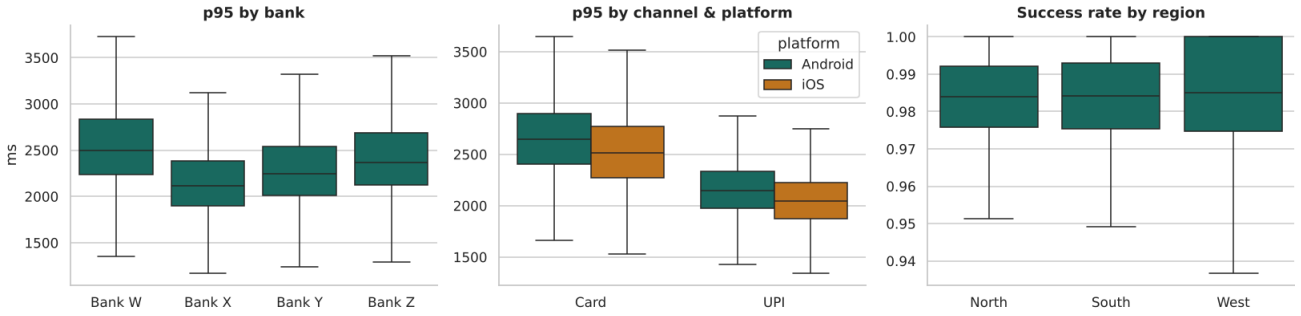



```
order = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
tot = hist.groupby("ts", observed=True).agg(volume=("volume", "sum"))
tot["hour"], tot["weekday"] = tot.index.hour, tot.index.day_name().str[:3]
heat = tot.pivot_table(index="weekday", columns="hour", values="volume",
aggfunc="mean").loc[order]
fig, ax = plt.subplots(1, 2, figsize=(15, 3.8), gridspec_kw={"width_ratios": [1.6,
1]})
sns.heatmap(heat, cmap="YlGnBu", ax=ax[0], cbar_kws={"label": "payments / 5
min"}); ax[0].set(title="Seasonality: average volume by weekday × hour",
ylabel="")
prof = hist.groupby(["is_weekend", "hour"],
observed=True).p95_ms.mean().unstack(0)
ax[1].plot(prof.index, prof[0], color=TEAL, label="weekday");
ax[1].plot(prof.index, prof[1], color=AMBER, label="weekend")
ax[1].set(title="Average p95 payment time by hour", xlabel="hour", ylabel="ms");
ax[1].legend()
plt.tight_layout(); plt.show()
peak, trough = heat.stack().idxmax(), heat.stack().idxmin()
RESULTS["peak_trough_ratio"] = heat.values.max() / heat.values.min()
say(f"**Busiest time:** {peak[0]} {peak[1]}:00 · **quietest:** {trough[0]}
{trough[1]}:00 · ratio **{RESULTS['peak_trough_ratio']:.1f}×**. "
    "A single fixed threshold cannot be right for both, which is the core reason
Lumio learns an *expected* value for every moment.")
```

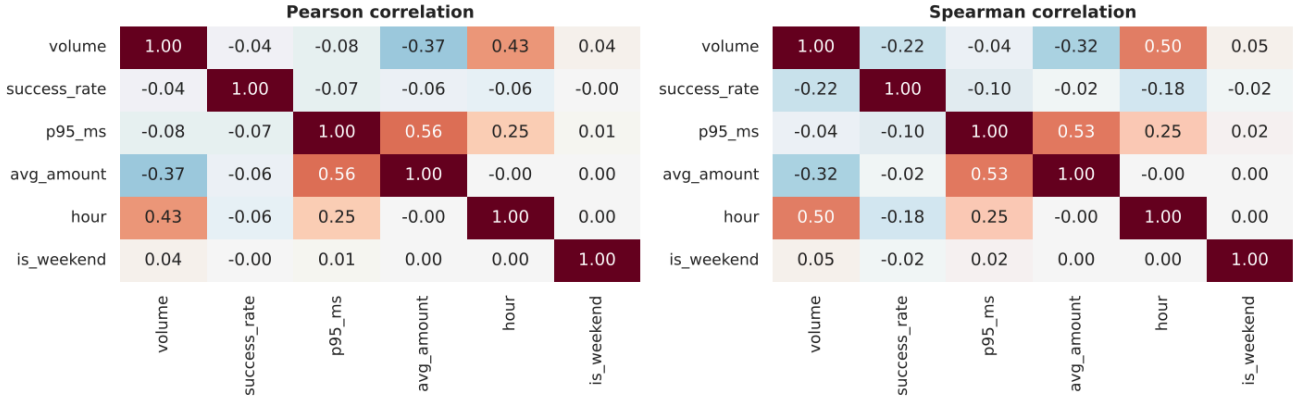


Busiest time: Fri 19:00 · **quietest:** Tue 3:00 · ratio **18.9×**. A single fixed threshold cannot be right for both, which is the core reason Lumio learns an *expected* value for every moment.

```
fig, ax = plt.subplots(1, 3, figsize=(15, 3.8))
sns.boxplot(data=hist, x="bank", y="p95_ms", ax=ax[0], color=TEAL,
showfliers=False); ax[0].set(title="p95 by bank", xlabel="", ylabel="ms")
sns.boxplot(data=hist, x="channel", y="p95_ms", hue="platform", ax=ax[1], palette=
[TEAL, AMBER], showfliers=False); ax[1].set(title="p95 by channel & platform",
xlabel="", ylabel="")
sns.boxplot(data=hist, x="region", y="success_rate", ax=ax[2], color=TEAL,
showfliers=False); ax[2].set(title="Success rate by region", xlabel="", ylabel="")
plt.tight_layout(); plt.show()
```




```
cols = ["volume", "success_rate", "p95_ms", "avg_amount", "hour", "is_weekend"]
fig, ax = plt.subplots(1, 2, figsize=(13, 4.2))
for a, m in zip(ax, ["pearson", "spearman"]):
    sns.heatmap(hist[cols].corr(method=m), annot=True, fmt=".2f", cmap="RdBu_r",
center=0, vmin=-1, vmax=1, ax=a, cbar=False)
    a.set(title=f"{m.title()} correlation")
plt.tight_layout(); plt.show()
```




```
# Upstream services: normal range in history
sh = svc[svc.ts < "2025-02-07"]
display(sh.groupby("service").agg(p95_median=("p95_ms", "median"), p95_p99=
("p95_ms", lambda s: s.quantile(.99)),
                                  error_rate_mean=("error_rate",
"mean")).round(4))
```

service	p95_median	p95_p99	error_rate_mean
card-network	267.20	336.013	0.002
core-banking-W	333.80	421.248	0.002
core-banking-X	333.80	423.306	0.002
core-banking-Y	333.70	418.368	0.002
core-banking-Z	334.30	415.859	0.002
fraud-check	133.60	167.865	0.002
upi-switch	200.35	252.973	0.002

EDA findings

- **Strong daily and weekly seasonality** in volume (the peak-to-trough ratio above), so "normal" must be time-aware.
- **Card is slower than UPI** and carries higher payment values. Android is slightly slower than iOS. Bank W is the slowest bank.
- p95 is **right-skewed** (skew > 0), so we model **log(p95)** in regression.

- Success rate is very stable (~98 %), so small drops matter and need *statistical* rather than eyeball thresholds.
- `avg_amount` correlates with p95 overall (≈ 0.56), but is this causal? Section 6.1 shows it is **confounding** by channel.

---

4. SQL analysis

The cleaned data is loaded into a **SQLite** database (the live app uses SQLite or PostgreSQL). These queries are the kind an analyst writes daily.

```
con = sqlite3.connect(":memory:")
cols = ["ts", "bank", "channel", "platform", "region", "volume", "success_count",
"failed", "p95_ms", "avg_amount", "period"]
t = pay[cols].copy(); t["ts"] = t.ts.dt.strftime("%Y-%m-%d %H:%M:%S")
for c in SEG: t[c] = t[c].astype(str)
t.to_sql("payments", con, index=False)
s = svc.copy(); s["ts"] = s.ts.dt.strftime("%Y-%m-%d %H:%M:%S");
s.to_sql("services", con, index=False)
con.execute("CREATE INDEX ix_p ON payments(ts, bank, channel)");
con.execute("CREATE INDEX ix_s ON services(ts, service)")
def q(sql): return pd.read_sql_query(sql, con)
print("Tables:", q("SELECT name FROM sqlite_master WHERE
type='table'").name.tolist(), "| SQLite", sqlite3.sqlite_version)
```

```
Tables: ['payments', 'services'] | SQLite 3.45.1
```

Q1 · Aggregation: success rate and payment time by bank and channel (history).

```
q("""
SELECT bank, channel,
       SUM(volume)                                    AS payments,
       ROUND(100.0 * SUM(success_count) / SUM(volume), 2) AS success_pct,
       ROUND(AVG(p95_ms), 0)                          AS avg_p95_ms
FROM payments
WHERE period = 'history'
GROUP BY bank, channel
ORDER BY avg_p95_ms DESC
""")
```

	bank	channel	payments	success_pct	avg_p95_ms
0	Bank W	Card	2526902	98.09	2818.0
1	Bank Z	Card	3033137	98.08	2673.0
2	Bank Y	Card	4040453	98.09	2527.0
3	Bank X	Card	5057465	98.08	2381.0
4	Bank W	UPI	4595516	98.28	2284.0
5	Bank Z	UPI	5510520	98.28	2167.0
6	Bank Y	UPI	7350552	98.29	2051.0
7	Bank X	UPI	9188879	98.28	1932.0

Q2 · CASE + date functions: average hourly volume, business days vs weekends.

```
q("""
SELECT CAST(strftime('%H', ts) AS INT)                                    AS hour,
       ROUND(AVG(CASE WHEN strftime('%w', ts) IN ('0','6') THEN NULL ELSE vol
END)) AS business_day,
       ROUND(AVG(CASE WHEN strftime('%w', ts) IN ('0','6') THEN vol END))
AS weekend
FROM (SELECT ts, SUM(volume) AS vol FROM payments WHERE period='history' GROUP BY
ts)
GROUP BY hour ORDER BY hour
""").set_index("hour").T
```

hour	0	1	2	3	4	5	6	7	8	9	10	11	12	
business_day	3765.0	2267.0	1455.0	1041.0	986.0	1648.0	4245.0	7419.0	8721.0	8065.0	7229.0	7325.0	7567.0	76
weekend	10272.0	8605.0	6867.0	5109.0	2930.0	1475.0	1585.0	2278.0	3772.0	5636.0	7237.0	8520.0	9351.0	94

Q3 · Window function RANK(): the 3 segments with the most failed payments each day of the live period.

```
q("""
WITH daily AS (
    SELECT date(ts) AS day, bank || ' · ' || channel || ' · ' || platform || ' · '
|| region AS segment,
           SUM(failed) AS failed, SUM(volume) AS volume
    FROM payments WHERE period = 'live'
    GROUP BY day, segment)
SELECT day, segment, failed, ROUND(100.0*failed/volume, 2) AS fail_pct, rnk
FROM (SELECT *, RANK() OVER (PARTITION BY day ORDER BY failed DESC) AS rnk FROM
daily)
WHERE rnk <= 3
ORDER BY day, rnk
""").head(15)
```

	day	segment	failed	fail_pct	rnk
0	2025-02-07	Bank X · UPI · Android · North	2627	2.22	1
1	2025-02-07	Bank X · UPI · Android · South	2407	2.32	2
2	2025-02-07	Bank X · UPI · Android · West	1723	2.33	3
3	2025-02-08	Bank Y · Card · Android · North	6432	10.69	1
4	2025-02-08	Bank Y · Card · Android · South	5607	10.61	2
5	2025-02-08	Bank Y · Card · Android · West	3985	10.56	3
6	2025-02-09	Bank X · UPI · Android · North	1797	1.72	1
7	2025-02-09	Bank X · UPI · Android · South	1496	1.62	2
8	2025-02-09	Bank Y · UPI · Android · North	1403	1.67	3
9	2025-02-10	Bank X · UPI · Android · North	1098	1.63	1
10	2025-02-10	Bank X · UPI · Android · South	938	1.59	2
11	2025-02-10	Bank Y · UPI · Android · North	896	1.66	3
12	2025-02-11	Bank X · UPI · Android · North	859	1.54	1
13	2025-02-11	Bank X · UPI · Android · South	773	1.58	2
14	2025-02-11	Bank X · UPI · Android · West	732	2.09	3

Q4 · JOIN: card payments vs the card-network service during the Friday morning outage. The join reveals the cause.

```
q("""
SELECT strftime('%H:%M', p.ts) AS time,
       ROUND(AVG(p.p95_ms))                                  AS card_payment_p95_ms,
       ROUND(100.0 * SUM(p.failed) / SUM(p.volume), 2)       AS card_fail_pct,
       s.p95_ms                                              AS card_network_p95_ms,
       ROUND(100 * s.error_rate, 2)                          AS card_network_err_pct
FROM payments p
JOIN services s ON s.ts = p.ts AND s.service = 'card-network'
WHERE p.channel = 'Card' AND p.ts BETWEEN '2025-02-07 09:50:00' AND '2025-02-07
11:20:00'
  AND strftime('%M', p.ts) IN ('00','10','20','30','40','50')
GROUP BY p.ts ORDER BY p.ts
""")
```

	time	card_payment_p95_ms	card_fail_pct	card_network_p95_ms	card_network_err_pct
0	09:50	2908.0	1.95	268.3	0.30
1	10:00	2680.0	2.19	264.0	0.25
2	10:10	2916.0	1.91	291.2	0.19
3	10:20	6640.0	8.02	769.1	6.52
4	10:30	6669.0	6.83	816.5	6.49
5	10:40	8097.0	7.03	824.9	6.42
6	10:50	7029.0	7.29	812.3	6.54
7	11:00	2764.0	1.86	283.6	0.10
8	11:10	2952.0	1.84	286.0	0.26
9	11:20	2623.0	1.95	261.4	0.16

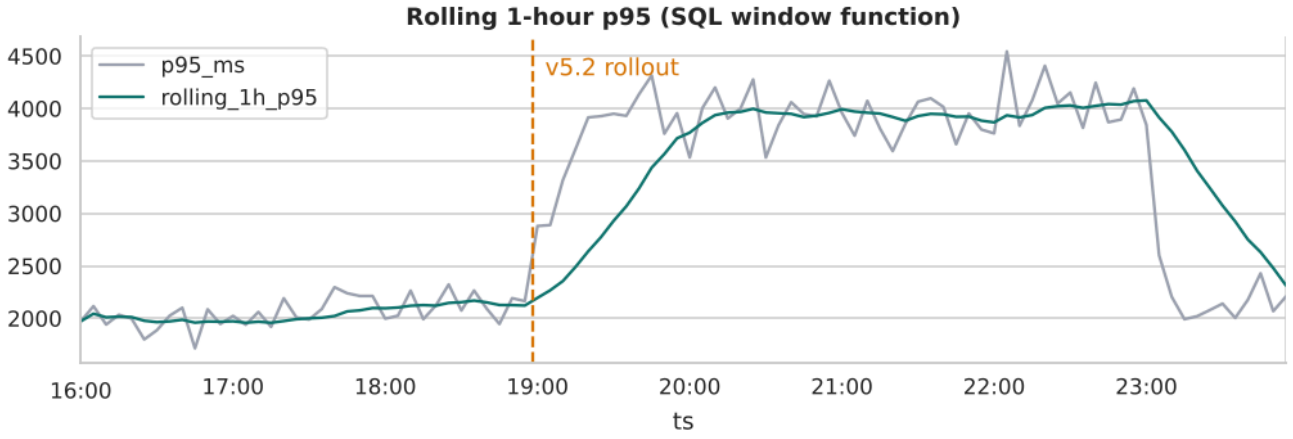
Q5 · CTE + LAG(): day-over-day change in total volume. The storm (Mon 10 – Tue 11) stands out.

```
q("""
WITH d AS (SELECT date(ts) AS day, SUM(volume) AS volume FROM payments GROUP BY
day)
SELECT day, volume,
       LAG(volume) OVER (ORDER BY day)                                        AS
prev_day,
       ROUND(100.0 * (volume - LAG(volume, 7) OVER (ORDER BY day)) / LAG(volume,
7) OVER (ORDER BY day), 1) AS vs_same_day_last_week_pct
FROM d WHERE day >= '2025-02-03' ORDER BY day
""")
```

	day	volume	prev_day	vs_same_day_last_week_pct
0	2025-02-03	1613014	NaN	NaN
1	2025-02-04	1811465	1613014.0	NaN
2	2025-02-05	1932156	1811465.0	NaN
3	2025-02-06	2027888	1932156.0	NaN
4	2025-02-07	2145519	2027888.0	NaN
5	2025-02-08	2237706	2145519.0	NaN
6	2025-02-09	1901450	2237706.0	NaN
7	2025-02-10	1217007	1901450.0	-24.6
8	2025-02-11	1012584	1217007.0	-44.1
9	2025-02-12	1812014	1012584.0	-6.2
10	2025-02-13	1981562	1812014.0	-2.3
11	2025-02-14	2597773	1981562.0	21.1
12	2025-02-15	2319552	2597773.0	3.7
13	2025-02-16	1903935	2319552.0	0.1

Q6 · Rolling window AVG() OVER (ROWS …): 1-hour rolling payment time for Bank X · UPI · Android on Friday evening, when app v5.2 rolls out at 18:58.

```
r = q("""
SELECT ts, ROUND(p95, 0) AS p95_ms,
       ROUND(AVG(p95) OVER (ORDER BY ts ROWS BETWEEN 11 PRECEDING AND CURRENT
ROW), 0) AS rolling_1h_p95
FROM (SELECT ts, SUM(p95_ms*volume)/SUM(volume) AS p95 FROM payments
      WHERE bank='Bank X' AND channel='UPI' AND platform='Android' AND ts BETWEEN
'2025-02-07 16:00:00' AND '2025-02-07 23:55:00'
      GROUP BY ts)
""", ); r["ts"] = pd.to_datetime(r.ts)
ax = r.plot(x="ts", y=["p95_ms", "rolling_1h_p95"], color=[GREY, TEAL], figsize=
(11, 3), title="Rolling 1-hour p95 (SQL window function)")
ax.axvline(pd.Timestamp("2025-02-07 18:58"), color=AMBER, ls="--");
ax.text(pd.Timestamp("2025-02-07 19:03"), r.p95_ms.max()*.95, "v5.2 rollout",
color=AMBER); plt.show()
```



---

5. Hypothesis testing

Each test states H_0 / H_1, checks assumptions, reports the p-value **and an effect size**. With hundreds of thousands of rows almost *any* difference is "statistically significant", so the effect size tells us whether it **matters**.

To reduce dependence between consecutive 5-minute rows, tests 5.1–5.3 use **segment × hour averages** (independent-enough units, n ≈ 24 k).

```
hh = (hist.assign(hr=hist.ts.dt.floor("h"))
          .groupby(["hr"] + SEG, observed=True)
          .agg(p95_ms=("p95_ms", "mean"), volume=("volume", "sum"), success_count=
("success_count", "sum"), failed=("failed", "sum"))
          .reset_index())
def cohens_d(a, b): return (a.mean() - b.mean()) / np.sqrt((a.var(ddof=1) +
b.var(ddof=1)) / 2)
def verdict(p, alpha=.05): return "**reject H₀**" if p < alpha else "**fail to
reject H₀**"
print(f"Hourly units: {len(hh):,}")
```

```
Hourly units: 24,192
```

5.1 Two-sample t-test (Welch) — Android vs iOS payment time

H_0: μ_Android = μ_iOS · H_1: μ_Android ≠ μ_iOS · α = 0.05

```
a, b = hh[hh.platform == "Android"].p95_ms, hh[hh.platform == "iOS"].p95_ms
t, p = stats.ttest_ind(a, b, equal_var=False)
lev = stats.levene(a, b)
d = cohens_d(a, b)
RESULTS["t_android_ios"] = (a.mean() - b.mean(), p, d)
say(f"Android mean **{a.mean():.0f} ms**, iOS **{b.mean():.0f} ms** (difference
{a.mean()-b.mean():+.0f} ms). "
    f"Levene test for equal variances p = {lev.pvalue:.3g} → Welch's t-test (does
not assume equal variances).  \n"
    f"t = {t:.2f}, p = {p:.3g} → {verdict(p)}. Cohen's d = **{d:.2f}** "
    f"({'small' if abs(d) < .5 else 'medium' if abs(d) < .8 else 'large'}
effect).")
```

Android mean **2413 ms**, iOS **2295 ms** (difference +118 ms). Levene test for equal variances p = 6.42e-07 → Welch's t-test (does not assume equal variances).
t = 26.28, p = 4.05e-150 → **reject H_0**. Cohen's d = **0.34** (small effect).

5.2 Chi-square test of independence — does payment outcome depend on channel?

H_0: success/failure is independent of channel.

```
ct = hist.groupby("channel", observed=True)[["success_count", "failed"]].sum()
chi2, p, dof, exp = stats.chi2_contingency(ct)
n = ct.values.sum(); v = np.sqrt(chi2 / (n * (min(ct.shape) - 1)))
display(ct.assign(fail_pct=lambda d: 100 * d.failed / (d.failed +
d.success_count)).round(3))
RESULTS["chi_channel"] = (p, v)
say(f"χ² = {chi2:,.1f}, dof = {dof}, p = {p:.3g} → {verdict(p)}. **Cramér's V =
{v:.4f}** (negligible).  \n"
    "**Lesson: statistically significant ≠ practically significant.** With ~40
```

```
million payments even a tiny difference is 'significant'. "
    "The effect size shows it is too small to matter for operations.")
```

channel	success_count	failed	fail_pct
Card	14377033	280924	1.917
UPI	26188004	457463	1.717

χ^2 = 2,147.3, dof = 1, p = 0 → **reject H_0. Cramér's V = 0.0072** (negligible).
Lesson: statistically significant ≠ practically significant. With ~40 million payments even a tiny difference is 'significant'. The effect size shows it is too small to matter for operations.

5.3 One-way ANOVA + Tukey HSD + Kruskal-Wallis — payment time across the four banks

H_0: μ_W = μ_X = μ_Y = μ_Z. ANOVA says *whether* some mean differs; Tukey HSD says *which pairs*; Kruskal-Wallis is the non-parametric check (no normality assumption).

```
groups = [g.p95_ms.values for _, g in hh.groupby("bank", observed=True)]
F, p = stats.f_oneway(*groups)
H, p_kw = stats.kruskal(*groups)
ss_between = sum(len(g) * (g.mean() - hh.p95_ms.mean())**2 for g in groups);
ss_total = ((hh.p95_ms - hh.p95_ms.mean())**2).sum()
eta2 = ss_between / ss_total
RESULTS["anova_bank"] = (p, eta2)
eta_lbl = "large" if eta2 >= .14 else "medium" if eta2 >= .06 else "small" if eta2
>= .01 else "negligible"
say(f"ANOVA F = {F:,.1f}, p = {p:.3g} → {verdict(p)}; **η² = {eta2:.3f}**, a **
{eta_lbl}** effect (bank explains {eta2:.1%} of the variation in hourly p95;
Cohen's guide: 0.01 small, 0.06 medium, 0.14 large).  \n"
    f"Kruskal-Wallis H = {H:,.1f}, p = {p_kw:.3g}: same conclusion without
assuming normality.")
tk = pairwise_tukeyhsd(hh.p95_ms, hh.bank.astype(str))
display(pd.DataFrame(tk.summary().data[1:], columns=tk.summary().data[0]))
```

ANOVA F = 1,694.1, p = 0 → **reject H_0**; **η^2 = 0.174**, a **large** effect (bank explains 17.4% of the variation in hourly p95; Cohen's guide: 0.01 small, 0.06 medium, 0.14 large).
Kruskal-Wallis H = 4,034.1, p = 0: same conclusion without assuming normality.

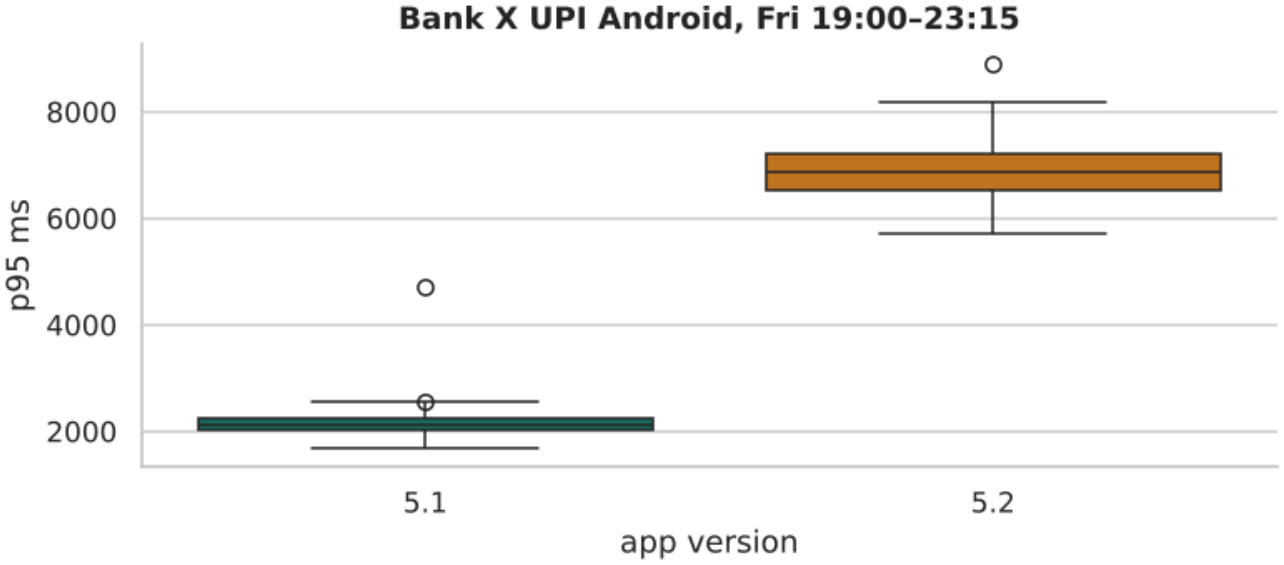
	group1	group2	meandiff	p-adj	lower	upper	reject
0	Bank W	Bank X	-395.0818	0.0	-410.0816	-380.0820	True
1	Bank W	Bank Y	-262.7440	0.0	-277.7438	-247.7442	True
2	Bank W	Bank Z	-131.8112	0.0	-146.8110	-116.8114	True
3	Bank X	Bank Y	132.3378	0.0	117.3381	147.3376	True
4	Bank X	Bank Z	263.2706	0.0	248.2708	278.2704	True
5	Bank Y	Bank Z	130.9328	0.0	115.9330	145.9326	True

5.4 Root-cause test, as Lumio does it: is app v5.2 really slower than v5.1?

Live period, Bank X · UPI · Android, Friday 19:00–23:15, **both versions running side by side** (a natural A/B test).

- **Mann-Whitney U** on payment time (non-parametric: p95 values are skewed)
- **Fisher's exact test** on failures (2×2 table)

```
w = demo_raw[(demo_raw.bank == "Bank X") & (demo_raw.channel == "UPI") &
(demo_raw.platform == "Android")
             & (demo_raw.ts >= "2025-02-07 19:00") & (demo_raw.ts < "2025-02-07
23:15")]
v51, v52 = w[w.app_version == 5.1].p95_ms, w[w.app_version == 5.2].p95_ms
U, p_mw = stats.mannwhitneyu(v52, v51, alternative="greater")
tab = w.assign(failed=w.volume - w.success_count).groupby("app_version")
[["failed", "success_count"]].sum().loc[[5.2, 5.1]]
tab["fail %"] = (100 * tab.failed / (tab.failed + tab.success_count)).round(2)
odds, p_f = stats.fisher_exact(tab[["failed", "success_count"]].values)
RESULTS["release_ratio"] = v52.median() / v51.median()
fig, ax = plt.subplots(figsize=(8, 3))
sns.boxplot(x=w.app_version.astype(str), y=w.p95_ms, palette=[TEAL, AMBER],
ax=ax); ax.set(title="Bank X UPI Android, Fri 19:00–23:15", xlabel="app version",
ylabel="p95 ms"); plt.show()
display(tab)
say(f"Median p95: v5.1 **{v51.median():.0f} ms** vs v5.2 **{v52.median():.0f} ms**
→ **{RESULTS['release_ratio']:.1f}× slower**.  \n"
    f"Mann-Whitney U = {U:,.0f}, one-sided p = {p_mw:.3g} → {verdict(p_mw)}.  \n"
    f"Fisher exact on failures: v5.2 failure rate {tab['fail %'].iloc[0]:.1f} % vs
{tab['fail %'].iloc[1]:.1f} %, odds ratio = **{odds:.1f}**, p = {p_f:.3g} →
{verdict(p_f)}.  \n"
    "This is exactly the check Lumio runs before it tells Release Management
*'v5.2 is the cause'*. The claim is only made if the test passes.")
```



app_version	failed	success_count	fail %
5.2	2161	27248	7.35
5.1	918	50406	1.79

Median p95: v5.1 **2124 ms** vs v5.2 **6878 ms** → **3.2× slower**.
Mann-Whitney U = 22,950, one-sided p = 1.76e-51 → **reject H_0**.
Fisher exact on failures: v5.2 failure rate 7.3 % vs 1.8 %, odds ratio = **4.4**, p = 0 → **reject H_0**.
This is exactly the check Lumio runs before it tells Release Management *'v5.2 is the cause'*.
The claim is only made if the test passes.

5.5 Confidence intervals — why a quiet segment needs more evidence

Same success rate, very different certainty. Wilson 95 % intervals for one segment at 4 am vs 6 pm.

```
seg = hist[(hist.segment == "Bank Y · UPI · iOS · South")]
rows = []
for label, hr in [("04:00 (quiet)", 4), ("18:00 (busy)", 18)]:
    r = seg[seg.hour == hr].iloc[0]
    lo, hi = proportion_confint(r.success_count, r.volume, method="wilson")
    rows.append({"bucket": label, "payments": int(r.volume), "success rate":
r.success_rate, "95% CI low": lo, "95% CI high": hi, "CI width (pts)": 100 * (hi -
lo)})
display(pd.DataFrame(rows).round(4))
say("With only a few payments the interval is many points wide: a couple of
failures prove nothing. "
    "This is why Lumio scales its z-scores by sampling noise and ignores segments
with fewer than 30 payments for success-rate alerts.")
```

	bucket	payments	success rate	95% CI low	95% CI high	CI width (pts)
0	04:00 (quiet)	29	0.8621	0.6944	0.9450	25.0616
1	18:00 (busy)	252	0.9881	0.9656	0.9959	3.0353

With only a few payments the interval is many points wide: a couple of failures prove nothing. This is why Lumio scales its z-scores by sampling noise and ignores segments with fewer than 30 payments for success-rate alerts.

---

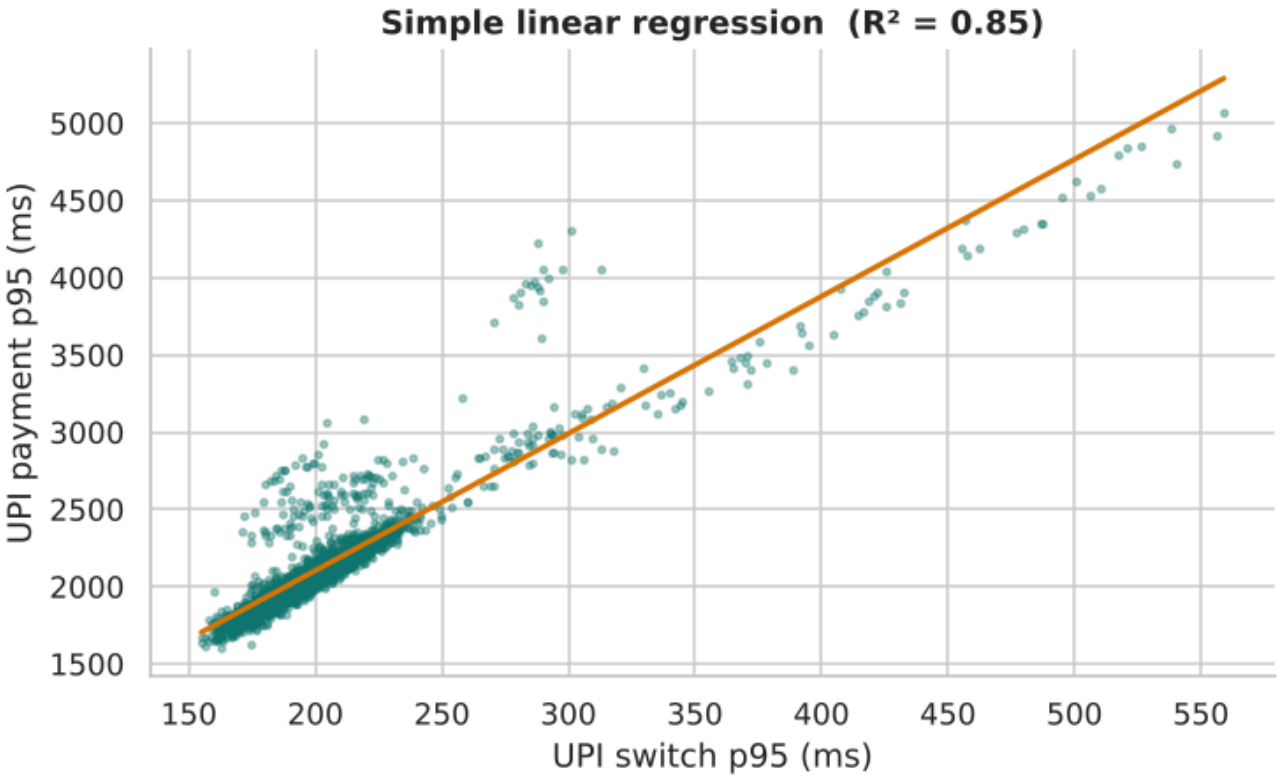
6. Linear regression

6.1 Simple linear regression: UPI payment time vs the UPI switch

$y = \beta_0 + \beta_1 \cdot x + \varepsilon$. Live period, 5-minute averages. Does the upstream service drive what customers feel?

```
u = (live[live.channel == "UPI"].groupby("ts").apply(lambda g:
np.average(g.p95_ms, weights=g.volume)).rename("upi_payment_p95")
     .to_frame().join(svc[svc.service == "upi-
switch"].set_index("ts").p95_ms.rename("upi_switch_p95")).dropna())
slr = smf.ols("upi_payment_p95 ~ upi_switch_p95", data=u).fit()
b0, b1 = slr.params; ci = slr.conf_int().loc["upi_switch_p95"]
fig, ax = plt.subplots(figsize=(7.5, 4.2))
ax.scatter(u.upi_switch_p95, u.upi_payment_p95, s=6, alpha=.35, color=TEAL)
xs = np.linspace(u.upi_switch_p95.min(), u.upi_switch_p95.max(), 50); ax.plot(xs,
b0 + b1 * xs, color=AMBER, lw=2)
ax.set(title=f"Simple linear regression  (R² = {slr.rsquared:.2f})", xlabel="UPI
switch p95 (ms)", ylabel="UPI payment p95 (ms)"); plt.show()
RESULTS["slr"] = (b1, slr.rsquared)
say(f"**ŷ = {b0:.0f} + {b1:.2f}·x**  ·  R² = **{slr.rsquared:.3f}**  ·  r =
{np.sqrt(slr.rsquared):.2f}  ·  slope 95 % CI [{ci[0]:.2f}, {ci[1]:.2f}], p =
{slr.pvalues.iloc[1]:.3g}.  \n"
```

```
    f"Interpretation: every extra **1 ms** at the UPI switch adds about **{b1:.1f}
ms** to the customer's payment time. "
    f"The switch explains {slr.rsquared:.0%} of the variation, so it is a key
dependency to monitor.")
```



ŷ = 329 + 8.87·x · R^2 = **0.846** · r = 0.92 · slope 95 % CI [8.74, 9.01], p = 0.
Interpretation: every extra **1 ms** at the UPI switch adds about **8.9 ms** to the customer's payment time. The switch explains 85% of the variation, so it is a key dependency to monitor.

6.2 Confounding: does a bigger payment take longer?

`avg_amount` correlated ≈ 0.56 with p95 in the EDA. Add `channel` as a control and watch the effect disappear (**omitted-variable bias**).

```
smp = hist.sample(40_000, random_state=SEED)
m1 = smf.ols("p95_ms ~ avg_amount", data=smp).fit()
m2 = smf.ols("p95_ms ~ avg_amount + C(channel)", data=smp).fit()
display(pd.DataFrame({"model": ["p95 ~ amount", "p95 ~ amount + channel"],
                      "β(amount) ms per $": [m1.params["avg_amount"],
m2.params["avg_amount"]],
                      "p-value": [m1.pvalues["avg_amount"],
m2.pvalues["avg_amount"]],
                      "R²": [m1.rsquared, m2.rsquared]}).round(4))
say(f"Alone, each extra $1 seems to add **{m1.params['avg_amount']:.1f} ms**. Once
channel is controlled, it is **{m2.params['avg_amount']:.2f} ms**. "
    "Card payments are both larger *and* slower; amount itself is not the cause.
**Correlation is not causation**: always ask what else differs between the
groups.")
```

	model	β(amount) ms per $	p-value	R^2
0	p95 ~ amount	18.1421	0.000	0.3109
1	p95 ~ amount + channel	0.4725	0.483	0.3230

Alone, each extra $1 seems to add **18.1 ms**. Once channel is controlled, it is **0.47 ms**. Card payments are both larger *and* slower; amount itself is not the cause. **Correlation is not causation**: always ask what else differs between the groups.

6.3 Multiple linear regression: what drives payment time?

log(p95) = β_0 + Σ β·(bank, channel, platform, region, hour, weekend, log volume, amount) + ε, fitted on the **3-week history**. Using log(y) means each coefficient reads as an approximate **% change**: (e^β − 1) × 100.

```
formula = "log_p95 ~ C(bank) + C(channel) + C(platform) + C(region) + C(hour) +
is_weekend + log_volume + avg_amount"
mlr = smf.ols(formula, data=hist).fit()
RESULTS["mlr_r2_train"] = mlr.rsquared
coef = pd.DataFrame({"coef": mlr.params, "p": mlr.pvalues, "ci_low":
mlr.conf_int()[0], "ci_high": mlr.conf_int()[1]})
coef["effect_%"] = 100 * (np.exp(coef.coef) - 1); coef.loc["Intercept",
"effect_%"] = np.nan
show = coef[~coef.index.str.startswith("C(hour)")]
display(show.round(4))
say(f"R² = **{mlr.rsquared:.3f}**, adjusted R² = {mlr.rsquared_adj:.3f}, F-test p
= {mlr.f_pvalue:.3g}, n = {int(mlr.nobs):,}. "
    f"(23 hour coefficients hidden for space; they are plotted below.)")
hc = coef[coef.index.str.startswith("C(hour)")]["effect_%"]; hc.index =
[int(i.split("T.")[1].rstrip("]")) for i in hc.index]
ax = pd.concat([pd.Series({0: 0.0}), hc]).sort_index().plot(kind="bar",
color=TEAL, figsize=(11, 2.8), title="Hour-of-day effect on payment time vs
midnight (%)")
ax.set(xlabel="hour", ylabel="%"); plt.show()
```

	coef	p	ci_low	ci_high	effect_%
Intercept	7.5392	0.0000	7.5286	7.5498	NaN
C(bank)[T.Bank X]	-0.2266	0.0000	-0.2279	-0.2253	-20.2771
C(bank)[T.Bank Y]	-0.1480	0.0000	-0.1492	-0.1468	-13.7580
C(bank)[T.Bank Z]	-0.0683	0.0000	-0.0694	-0.0671	-6.5986
C(channel)[T.UPI]	-0.2566	0.0000	-0.2606	-0.2526	-22.6297
C(platform)[T.iOS]	-0.0087	0.0000	-0.0096	-0.0078	-0.8651
C(region)[T.South]	0.0115	0.0000	0.0106	0.0125	1.1616
C(region)[T.West]	0.0898	0.0000	0.0887	0.0908	9.3942
is_weekend	-0.0060	0.0000	-0.0069	-0.0052	-0.6019
log_volume	0.0860	0.0000	0.0851	0.0870	8.9828
avg_amount	0.0001	0.2426	-0.0001	0.0002	0.0088

R^2 = **0.637**, adjusted R^2 = 0.637, F-test p = 0, n = 290,304. (23 hour coefficients hidden for space; they are plotted below.)




```
eff = lambda n: coef.loc[n, "effect_%"]
say(f"""**Reading the coefficients (holding everything else constant):**
* **UPI** payments take **{-eff('C(channel)[T.UPI]'):.1f} % less time** than card
payments: card is the slower rail.
* **iOS** vs Android: **{eff('C(platform)[T.iOS]'):+.1f} %**: once channel, bank
and region are controlled, platform barely matters (compare with the raw gap in
5.1).
* **West** region vs North: **{eff('C(region)[T.West]'):+.1f} %**.
* Bank X vs Bank W: **{eff('C(bank)[T.Bank X]'):+.1f} %** (Bank W is the slowest
bank).
* Doubling the volume (load) changes p95 by **{100*
(2**coef.loc['log_volume','coef']-1):+.1f} %** (elasticity).
* Weekend: **{eff('is_weekend'):+.1f} %** · amount: **{eff('avg_amount'):+.3f} %
per $** (p = {coef.loc['avg_amount','p']:.2f}, not significant: confirms 6.2).""")
```

Reading the coefficients (holding everything else constant):

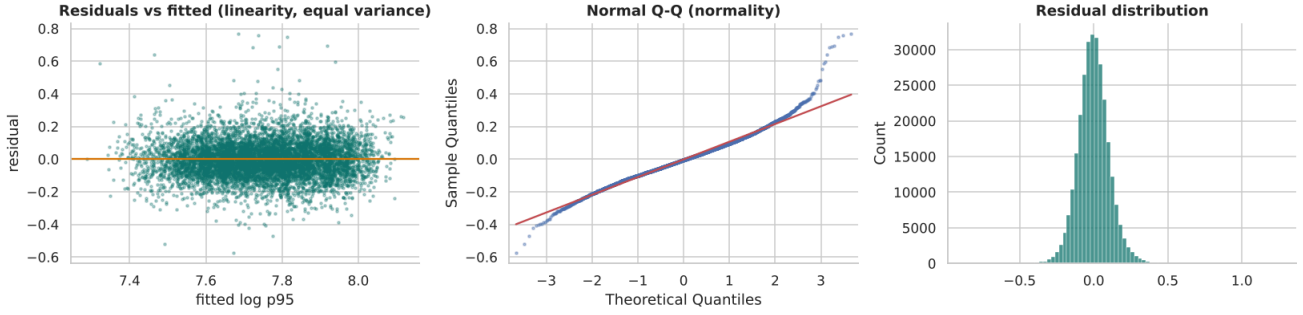
- **UPI** payments take **22.6 % less time** than card payments: card is the slower rail.
- **iOS** vs Android: **-0.9 %**: once channel, bank and region are controlled, platform barely matters (compare with the raw gap in 5.1).
- **West** region vs North: **+9.4 %**.
- Bank X vs Bank W: **-20.3 %** (Bank W is the slowest bank).
- Doubling the volume (load) changes p95 by **+6.1 %** (elasticity).
- Weekend: **-0.6 %** · amount: **+0.009 % per $** (p = 0.24, not significant: confirms 6.2).

6.4 Regression diagnostics: checking the assumptions (LINE: Linearity, Independence, Normality, Equal variance)

```
res, fit = mlr.resid, mlr.fittedvalues
idx = res.sample(8000, random_state=SEED).index
fig, ax = plt.subplots(1, 3, figsize=(15, 3.8))
ax[0].scatter(fit[idx], res[idx], s=4, alpha=.3, color=TEAL); ax[0].axhline(0,
color=AMBER); ax[0].set(title="Residuals vs fitted (linearity, equal variance)",
xlabel="fitted log p95", ylabel="residual")
sm.qqplot(res[idx], line="s", ax=ax[1], markersize=2, alpha=.4);
ax[1].set(title="Normal Q-Q (normality)")
sns.histplot(res, bins=80, ax=ax[2], color=TEAL); ax[2].set(title="Residual
distribution")
plt.tight_layout(); plt.show()

bp = het_breuschpagan(res[idx],
mlr.model.exog[mlr.model.data.row_labels.get_indexer(idx)])
```

```
dw = durbin_watson(res.sort_index())
X = hist[["log_volume", "avg_amount", "is_weekend", "hour"]].assign(card=
(hist.channel == "Card").astype(int), ios=(hist.platform == "iOS").astype(int))
Xc = sm.add_constant(X.sample(20000, random_state=SEED))
vif = pd.Series([variance_inflation_factor(Xc.values, i) for i in range(1,
Xc.shape[1])], index=Xc.columns[1:], name="VIF")
display(vif.round(2).to_frame().T)
say(f"* **Breusch-Pagan** p = {bp[1]:.3g} → {'heteroscedasticity present: use
robust (HC3) standard errors' if bp[1] < .05 else 'equal variance OK'}.  \n"
    f"* **Durbin-Watson** = {dw:.2f} ({'≈2: little autocorrelation' if 1.5 < dw <
2.5 else 'residuals are autocorrelated (time-series data), so p-values are
optimistic; Section 7 handles time explicitly'}).  \n"
    f"* **VIF** max = {vif.max():.1f} ({'no worrying multicollinearity (< 5)' if
vif.max() < 5 else 'multicollinearity: interpret with care'}). Note that channel
and amount are related, as 6.2 showed.  \n"
    f"* The log transform makes residuals close to symmetric; the Q-Q tails show
some heavy-tailed behaviour, typical of latency data.")
robust = mlr.get_robustcov_results("HC3")
print("Robust (HC3) SE for UPI vs Card:",
round(robust.bse[list(mlr.params.index).index("C(channel)[T.UPI]")], 5),
      "| classic SE:", round(mlr.bse["C(channel)[T.UPI]"], 5))
```



	log_volume	avg_amount	is_weekend	hour	card	ios
VIF	1.84	25.03	1.01	1.51	25.22	1.13

- **Breusch-Pagan** p = 5.84e-82 → heteroscedasticity present: use robust (HC3) standard errors.
- **Durbin-Watson** = 1.35 (residuals are autocorrelated (time-series data), so p-values are optimistic; Section 7 handles time explicitly).
- **VIF** max = 25.2 (multicollinearity: interpret with care). Note that channel and amount are related, as 6.2 showed.
- The log transform makes residuals close to symmetric; the Q-Q tails show some heavy-tailed behaviour, typical of latency data.

```
Robust (HC3) SE for UPI vs Card: 0.00215 | classic SE: 0.00203
```

6.5 Out-of-sample evaluation + regression as an anomaly detector

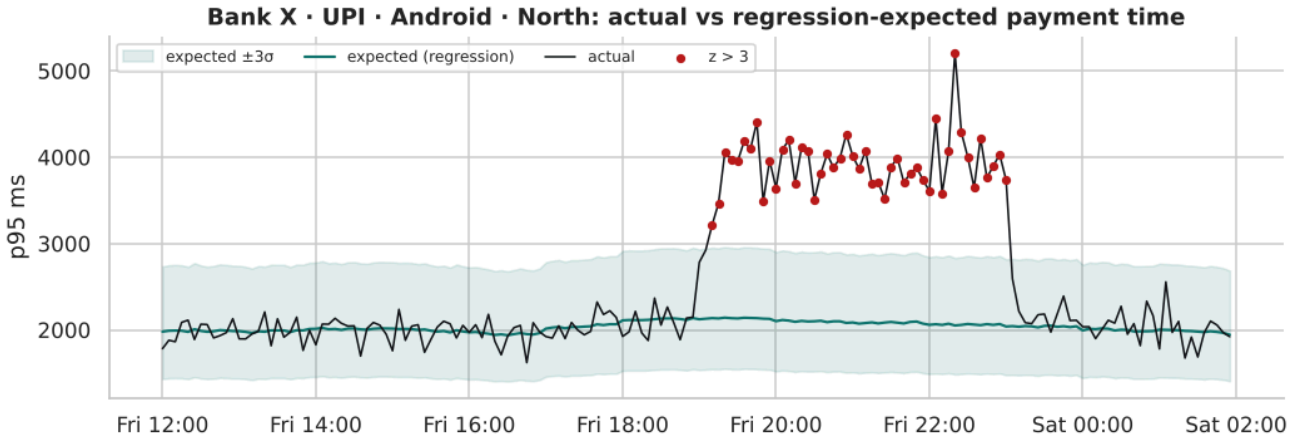
Train = history; **test = live period, normal rows only** (rows touched by a planted event are excluded, so we measure how well the model knows *normal*).

```
test = live[~live.any_event]
pred_ms = np.exp(mlr.predict(test))
base_ms = test.channel.map(hist.groupby("channel",
observed=True).p95_ms.mean()).astype(float)
```

```
ev = pd.DataFrame({
    "model": ["Baseline: channel average", "Multiple linear regression"],
    "MAE ms": [mean_absolute_error(test.p95_ms, base_ms),
mean_absolute_error(test.p95_ms, pred_ms)],
    "RMSE ms": [mean_squared_error(test.p95_ms, base_ms) ** .5,
mean_squared_error(test.p95_ms, pred_ms) ** .5],
    "R²": [r2_score(test.p95_ms, base_ms), r2_score(test.p95_ms,
pred_ms)]}).round(3)
display(ev); RESULTS["mlr_r2_test"] = ev.iloc[1]["R²"]; RESULTS["mlr_mae"] =
ev.iloc[1]["MAE ms"]

# residual z-score → anomaly
sigma = mlr.resid.std()
seg = live[(live.bank == "Bank X") & (live.channel == "UPI") & (live.platform ==
"Android") & (live.region == "North")
           & (live.ts >= "2025-02-07 12:00") & (live.ts < "2025-02-08
02:00")].copy()
seg["expected"] = np.exp(mlr.predict(seg)); seg["z"] = (seg.log_p95 -
mlr.predict(seg)) / sigma
fig, ax = plt.subplots(figsize=(11, 3.4))
ax.fill_between(seg.ts, seg.expected * np.exp(-3 * sigma), seg.expected * np.exp(3
* sigma), color=TEAL, alpha=.12, label="expected ±3σ")
ax.plot(seg.ts, seg.expected, color=TEAL, lw=1.4, label="expected (regression)");
ax.plot(seg.ts, seg.p95_ms, color=INK, lw=1, label="actual")
hit = seg[seg.z > 3]; ax.scatter(hit.ts, hit.p95_ms, color=RED, s=14, zorder=3,
label="z > 3")
ax.set(title="Bank X · UPI · Android · North: actual vs regression-expected
payment time", ylabel="p95 ms"); ax.legend(ncol=4, fontsize=8)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%a %H:%M")); plt.show()
say(f"First bucket with z > 3: **{hit.ts.min():%a %d %b %H:%M}** (the v5.2 rollout
started 18:58). The regression turns *'is this high?'* into *'is this higher than
expected **for this segment at this time**?'*")
```

	model	MAE ms	RMSE ms	R^2
0	Baseline: channel average	264.427	348.606	0.336
1	Multiple linear regression	192.339	268.383	0.606



First bucket with z > 3: **Fri 07 Feb 19:10** (the v5.2 rollout started 18:58). The regression turns *'is this high?'* into *'is this higher than expected **for this segment at this time**?'*

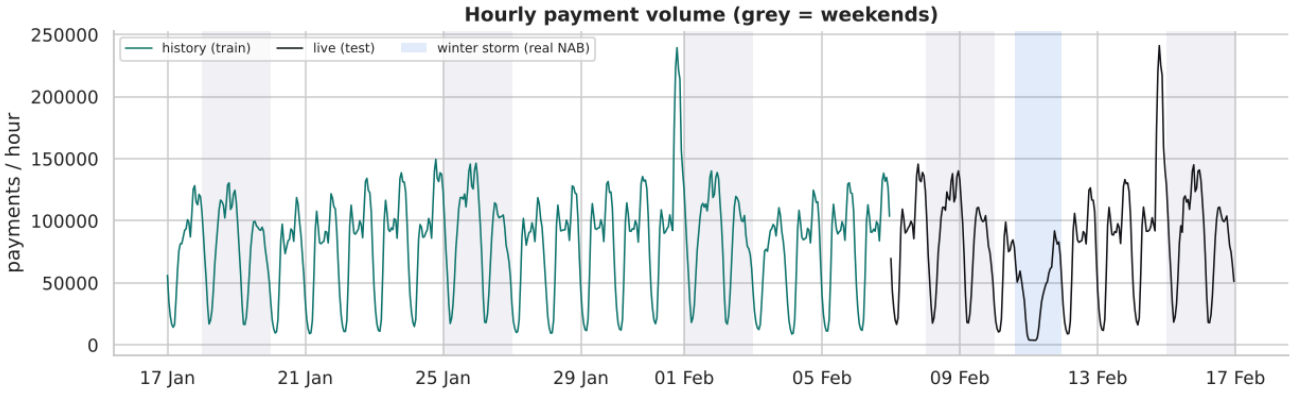
---

7. Time-series analysis & forecasting

Series: **total payments per hour**, all segments, 17 Jan → 16 Feb 2025 (744 hours). Train = 3-week history (504 h), test = the 10 live days (240 h).

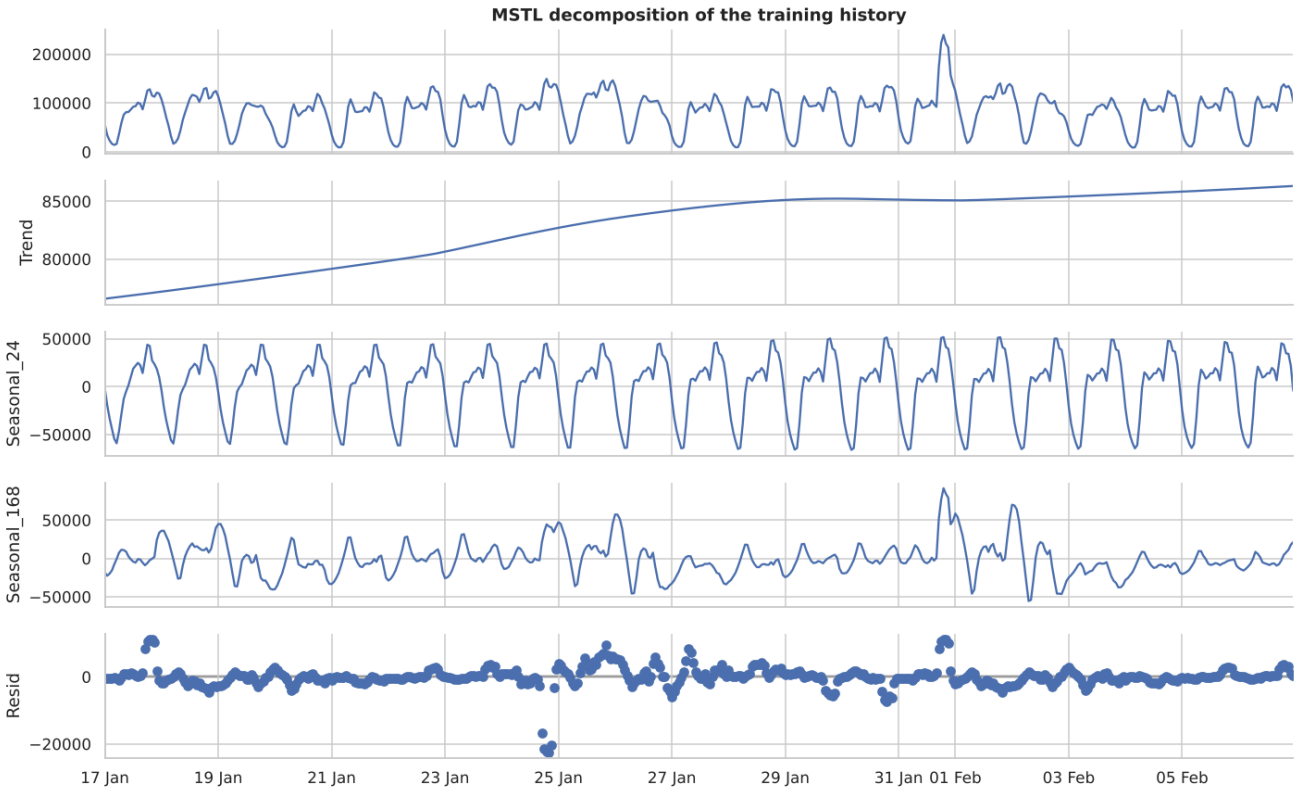
Walk-forward validation: every midnight of the test period we refit on everything known so far and forecast the next 24 hours, exactly how an operations team would use a forecast. This is more honest than one 240-hour forecast, and never lets the model see the future.

```
y = pay.groupby(pay.ts.dt.floor("h")).volume.sum().asfreq("h")
y_tr, y_te = y[:"2025-02-06 23:00"], y["2025-02-07":]
STORM = (pd.Timestamp("2025-02-10 14:00"), pd.Timestamp("2025-02-11 23:00"))
fig, ax = plt.subplots(figsize=(13, 3.6))
ax.plot(y_tr, color=TEAL, lw=1, label="history (train)"); ax.plot(y_te, color=INK,
lw=1, label="live (test)")
for d in pd.date_range(y.index[0], y.index[-1], freq="D"):
    if d.dayofweek >= 5: ax.axvspan(d, d + pd.Timedelta("1D"), color=GREY,
alpha=.12, lw=0)
ax.axvspan(*STORM, color=BLUE, alpha=.12, lw=0, label="winter storm (real NAB)")
ax.set(title="Hourly payment volume (grey = weekends)", ylabel="payments / hour");
ax.legend(ncol=3, fontsize=8)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d %b")); plt.show()
```



7.1 Decomposition (MSTL: trend + daily + weekly seasonality + remainder)

```
mstl = MSTL(y_tr, periods=(24, 168)).fit()
fig = mstl.plot(); fig.set_size_inches(13, 8)
for a in fig.axes: a.xaxis.set_major_formatter(mdates.DateFormatter("%d %b"))
fig.axes[0].set_title("MSTL decomposition of the training history",
fontweight="bold"); plt.tight_layout(); plt.show()
var = lambda s: np.var(s)
strength = {p: max(0, 1 - var(mstl.resid) / var(mstl.resid +
mstl.seasonal[f"seasonal_{p}"])) for p in (24, 168)}
RESULTS["seasonal_strength"] = strength
say(f"**Seasonal strength** (0–1): daily = **{strength[24]:.2f}**, weekly = **
{strength[168]:.2f}**. Both are strong: the series is dominated by predictable
rhythm.")
```



Seasonal strength (0–1): daily = **0.99**, weekly = **0.98**. Both are strong: the series is dominated by predictable rhythm.

7.2 Stationarity: ADF and KPSS

```
def stationarity(s, name):
    s = s.dropna()
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        adf_p = adfuller(s, autolag="AIC")[1]
        kpss_p = kpss(s, regression="c", nlags="auto")[1]
    concl = ("stationary" if adf_p < .05 and kpss_p >= .05 else "non-stationary"
if adf_p >= .05 and kpss_p < .05
             else "inconclusive (tests disagree)")
    return {"series": name, "ADF p  (H₀: unit root / non-stationary)":
round(adf_p, 4),
            "KPSS p  (H₀: stationary)": "≥ 0.10" if kpss_p >= .1 else
round(kpss_p, 4), "conclusion": concl}
ly = np.log(y)
st = pd.DataFrame([stationarity(ly[:"2025-02-06 23:00"], "log volume"),
                   stationarity(ly.diff(24)[:"2025-02-06 23:00"], "daily
difference (lag 24)"),
                   stationarity(ly.diff(168)[:"2025-02-06 23:00"], "weekly
difference (lag 168)")])
display(st)
say("ADF and KPSS have **opposite null hypotheses**, so agreement between them is
strong evidence. "
    f"Log volume is judged **{st.conclusion[0]}**: there is no wandering trend
(unit root). "
    "But stationarity in this sense does not mean 'no pattern': the mean still
depends on the hour and the weekday, as the ACF below shows. "
    "We remove that by **seasonal differencing at lag 168** (this hour minus the
same hour last week), which captures the daily *and* weekly rhythm in one step. "
    f"For the weekly-differenced series the tests say **{st.conclusion[2]}**: no
```

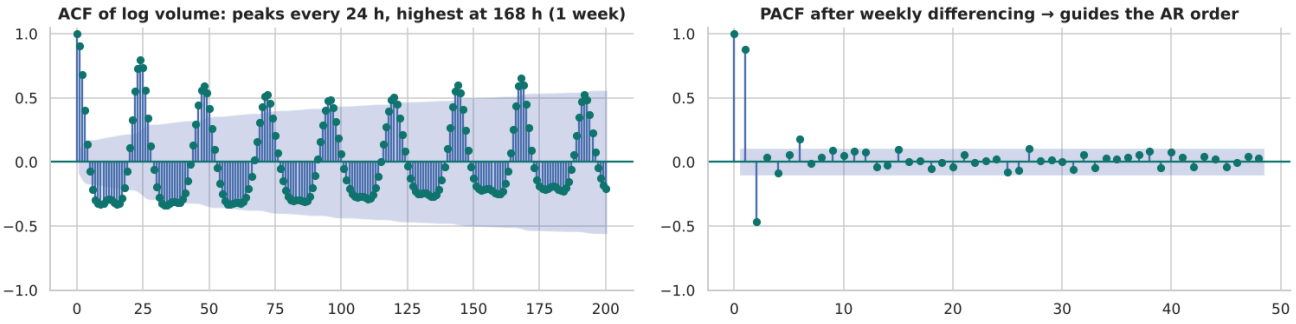
```
unit root, but some short-term autocorrelation remains, which is exactly what the
ARMA part of the model handles.")
```

	series	ADF p (H_0: unit root / non-stationary)	KPSS p (H_0: stationary)	conclusion
0	log volume	0.0	≥ 0.10	stationary
1	daily difference (lag 24)	0.0	≥ 0.10	stationary
2	weekly difference (lag 168)	0.0	0.01	inconclusive (tests disagree)

ADF and KPSS have **opposite null hypotheses**, so agreement between them is strong evidence. Log volume is judged **stationary**: there is no wandering trend (unit root). But stationarity in this sense does not mean 'no pattern': the mean still depends on the hour and the weekday, as the ACF below shows. We remove that by **seasonal differencing at lag 168** (this hour minus the same hour last week), which captures the daily *and* weekly rhythm in one step. For the weekly-differenced series the tests say **inconclusive (tests disagree)**: no unit root, but some short-term autocorrelation remains, which is exactly what the ARMA part of the model handles.

7.3 Autocorrelation (ACF) and partial autocorrelation (PACF)

```
z = ly - ly.shift(168)                          # weekly-differenced log volume
fig, ax = plt.subplots(1, 2, figsize=(14, 3.6))
plot_acf(ly["2025-02-06 23:00"], lags=200, ax=ax[0], color=TEAL);
ax[0].set(title="ACF of log volume: peaks every 24 h, highest at 168 h (1 week)")
plot_pacf(z["2025-02-06 23:00"].dropna(), lags=48, ax=ax[1], method="ywm",
color=TEAL); ax[1].set(title="PACF after weekly differencing → guides the AR
order")
for a in ax: a.set_ylim(-1, 1.05)
plt.tight_layout(); plt.show()
```



7.4 Model selection (AIC / BIC) and walk-forward forecasting

Model	Idea
Seasonal naive	"Same hour last week" (the benchmark any model must beat)
Holt-Winters (ETS)	Exponential smoothing of log volume with an additive weekly seasonal pattern
SARIMA(p,0,q)(0,1,0)$_{168}$ + payday	Weekly-differenced log volume, an ARMA model for what is left, and a **payday-evening regressor** (a known calendar event, so it is legitimate to tell the model about it in advance)

Choosing the ARMA order. We compute **AIC/BIC** (in-sample fit, penalised for complexity) *and* a **validation MAPE**: walk-forward day-ahead forecasts over the last training week (31 Jan – 6 Feb), still inside the history. Because our goal is *forecasting*, the order is chosen on validation error, so the live days remain an untouched test set.

```
pay_h = pd.Series((y.index.normalize().isin(pd.to_datetime(["2025-01-31", "2025-02-14"])) & (y.index.hour >= 15)).astype(float), index=y.index)
dpay = (pay_h - pay_h.shift(168)).fillna(0).rename("payday")   # the regressor, differenced the same way as y
z_tr = z[:"2025-02-06 23:00"].dropna()
def sarima_wf(order, days, alpha=.05):
    # walk-forward day-ahead SARIMA(order)(0,1,0)168 forecasts: refit every midnight on everything known so far
    pts, los, his = [], [], []
    for d in days:
        cut, idx = d - pd.Timedelta("1h"), pd.date_range(d, periods=24, freq="h"); zt = z[:cut].dropna()
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            f = ARIMA(zt, exog=dpay[zt.index], order=order, trend="n").fit().get_forecast(24, exog=dpay[idx])
        base = ly.shift(168).loc[idx].values; ci = f.conf_int(alpha=alpha).values
        pts.append(pd.Series(np.exp(f.predicted_mean.values + base), idx))
        los.append(pd.Series(np.exp(ci[:, 0] + base), idx)); his.append(pd.Series(np.exp(ci[:, 1] + base), idx))
    return pd.concat(pts), pd.concat(los), pd.concat(his)
val_days = pd.date_range("2025-01-31", "2025-02-06")
cands = [(1, 0, 0), (2, 0, 0), (3, 0, 0), (1, 0, 1), (2, 0, 1), (1, 0, 2)]
rows, fits = [], {}
for o in cands:
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        fits[o] = ARIMA(z_tr, exog=dpay[z_tr.index], order=o, trend="n").fit()
    pv = sarima_wf(o, val_days)[0]; av = y.loc[pv.index]
    rows.append({"ARMA order (p,d,q)": str(o), "AIC": fits[o].aic, "BIC": fits[o].bic, "validation MAPE %": 100 * np.mean(np.abs(av - pv) / av)})
sel = pd.DataFrame(rows).set_index("ARMA order (p,d,q)").round(2)
best_order = cands[int(np.argmin(sel["validation MAPE %"].values))]
aic_order = cands[int(np.argmin(sel["AIC"].values))]
sel["lowest AIC"] = ["✔" if o == aic_order else "" for o in cands]
sel["chosen (validation)"] = ["✔" if o == best_order else "" for o in cands]
display(sel)
SAR = f"SARIMA{best_order}(0,1,0)₁₆₈"
say(f"Chosen: **{SAR} + payday regressor**." + (
    f" Note that AIC prefers {aic_order}: AIC rewards *in-sample* fit, while validation measures what we care about, accuracy on days the model has not seen. "
    "When they disagree, trust the out-of-sample measure for forecasting." if aic_order != best_order else " AIC and validation agree."))
print(fits[best_order].summary().tables[1])
```

ARMA order (p,d,q)	AIC	BIC	validation MAPE %	lowest AIC	chosen (validation)
(1, 0, 0)	-1044.28	-1032.83	6.00		
(2, 0, 0)	-1121.93	-1106.66	6.13	✔	
(3, 0, 0)	-1120.92	-1101.83	6.08		
(1, 0, 1)	-1117.04	-1101.77	5.92		✔
(2, 0, 1)	-1121.53	-1102.45	6.02		
(1, 0, 2)	-1120.27	-1101.18	5.99		

Chosen: **SARIMA(1, 0, 1)(0,1,0)$_{168}$ + payday regressor**. Note that AIC prefers (2, 0, 0): AIC rewards *in-sample* fit, while validation measures what we care about, accuracy on days the model has not seen. When they disagree, trust the out-of-sample measure for forecasting.

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
payday         0.0466      0.018      2.533      0.011       0.011       0.083
ar.L1          0.8013      0.024     32.984      0.000       0.754       0.849
ma.L1          0.4905      0.035     13.922      0.000       0.421       0.560
sigma2         0.0020   7.38e-05     27.712      0.000       0.002       0.002
==============================================================================
```

```
t0 = time.time()
parts = {"Seasonal naive": [], "Holt-Winters": []}
for d in pd.date_range("2025-02-07", "2025-02-16"):
    cut, idx = d - pd.Timedelta("1h"), pd.date_range(d, periods=24, freq="h")
    parts["Seasonal naive"].append(y.shift(168).loc[idx])
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        hw = ExponentialSmoothing(ly[:cut], seasonal="add", seasonal_periods=168,
initialization_method="estimated").fit()
    parts["Holt-Winters"].append(pd.Series(np.exp(hw.forecast(24).values), idx))
fc = pd.DataFrame({k: pd.concat(v) for k, v in parts.items()})
fc[SAR], fc["lo"], fc["hi"] = sarima_wf(best_order, pd.date_range("2025-02-07",
"2025-02-16"))
fc["actual"] = y.loc[fc.index]
parts[SAR] = None
print(f"10 walk-forward days × 3 models fitted in {time.time()-t0:.0f}s")

normal = ~((fc.index >= STORM[0]) & (fc.index <= STORM[1]))
def mape(a, f): return 100 * np.mean(np.abs((a - f) / a))
acc = pd.DataFrame([{"model": m,
                     "MAE": mean_absolute_error(fc.actual[normal], fc[m][normal]),
                     "RMSE": mean_squared_error(fc.actual[normal], fc[m][normal])
** .5,
                     "MAPE %": mape(fc.actual[normal], fc[m][normal])} for m in
parts]).set_index("model").round(2)
acc["skill vs naive %"] = (100 * (1 - acc.MAE / acc.loc["Seasonal naive",
"MAE"])).round(1)
display(acc); RESULTS["forecast"] = acc
best = acc["MAPE %"].idxmin()
say(f"Accuracy on the **normal** live hours (the storm is excluded: no model
should predict a blizzard from payment history). "
    f"Best: **{best}**, MAPE **{acc.loc[best,'MAPE %']:.1f} %**, **
{acc.loc[best,'skill vs naive %']:.0f} %** lower error than the seasonal-naive
benchmark. "
    "Always compare against a naive benchmark: a complex model that cannot beat
'same hour last week' adds no value.")

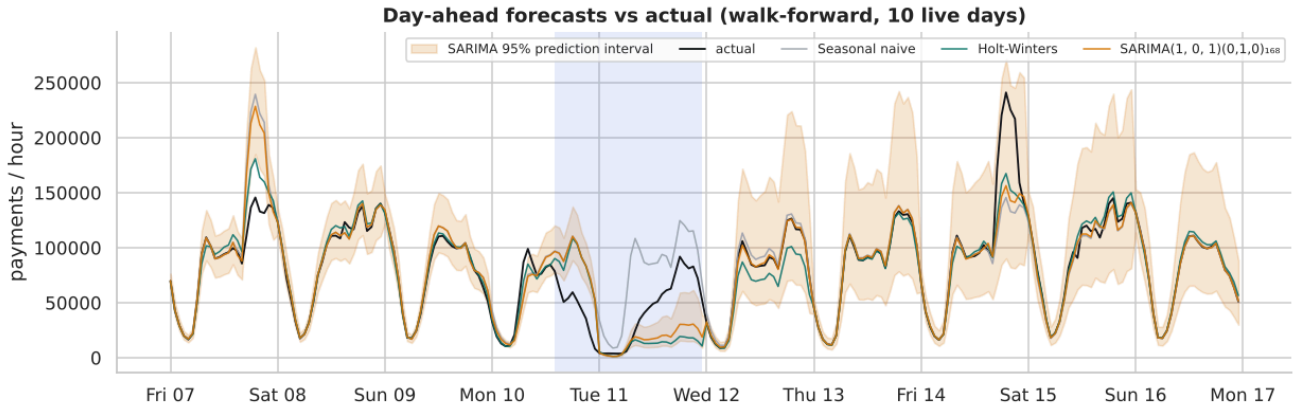
fig, ax = plt.subplots(figsize=(13, 3.8))
ax.fill_between(fc.index, fc.lo, fc.hi, color=AMBER, alpha=.18, label="SARIMA 95%
prediction interval")
ax.plot(fc.index, fc.actual, color=INK, lw=1.2, label="actual")
for m, c in [("Seasonal naive", GREY), ("Holt-Winters", TEAL), (SAR, AMBER)]:
ax.plot(fc.index, fc[m], color=c, lw=1, label=m, alpha=.9)
ax.axvspan(*STORM, color=BLUE, alpha=.10, lw=0)
ax.set(title="Day-ahead forecasts vs actual (walk-forward, 10 live days)",
```

```
ylabel="payments / hour"); ax.legend(ncol=5, fontsize=8)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%a %d")); plt.show()
```

```
10 walk-forward days × 3 models fitted in 4s
```

model	MAE	RMSE	MAPE %	skill vs naive %
Seasonal naive	7390.39	19330.89	7.52	0.0
Holt-Winters	6587.92	13275.25	6.23	10.9
SARIMA(1, 0, 1)(0,1,0)$_{168}$	6258.96	17099.86	5.89	15.3

Accuracy on the **normal** live hours (the storm is excluded: no model should predict a blizzard from payment history). Best: **SARIMA(1, 0, 1)(0,1,0)$_{168}$**, MAPE **5.9 %**, **15 %** lower error than the seasonal-naive benchmark. Always compare against a naive benchmark: a complex model that cannot beat 'same hour last week' adds no value.



7.5 Using the prediction interval to detect time-series anomalies

```
fc["outside"] = (fc.actual < fc.lo) | (fc.actual > fc.hi)
fc["in_storm"] = ~normal
display(pd.crosstab(fc.in_storm.map({True: "storm hours", False: "other hours"}),
fc.outside.map({True: "outside 95% PI", False: "inside"})))
storm_hit = fc[fc.in_storm].outside.mean(); other_hit =
fc[~fc.in_storm].outside.mean()
RESULTS["storm_flag"] = (storm_hit, other_hit)
say(f"**{storm_hit:.0%}** of storm hours fall outside the 95 % prediction
interval, vs **{other_hit:.0%}** of other hours "
    f"(≈ 5 % would be expected by chance). The storm is detected purely from the
time series. "
    "Lumio then checks *breadth* (every bank, every region, with healthy success
rates) and classifies it as **demand, not a fault**: "
    "one advisory to Business Operations instead of 48 pages.")
```

in_storm	outside	inside	outside 95% PI
other hours	193		13
storm hours	6		28

82% of storm hours fall outside the 95 % prediction interval, vs **6%** of other hours (≈ 5 % would be expected by chance). The storm is detected purely from the time series. Lumio then checks *breadth* (every bank, every region, with healthy success rates) and classifies it as **demand, not a fault**: one advisory to Business Operations instead of 48 pages.

8. Classification: logistic regression for incident detection

Unit: segment × 15-minute scan (the live app's scan interval). **Label:** 1 if the segment is inside a real incident window (from the answer key, used only for supervision and evaluation).

Features (all *expected vs actual*, so they are seasonal-aware):

Feature	Definition
`lat_z`	Regression residual of log p95 (Section 6) ÷ σ
`fail_z`	(expected success − actual success) ÷ binomial standard error
`vol_drop` , `vol_jump`	Negative / positive part of log(actual ÷ expected volume)

Split by time (never shuffle time-series data): train = Fri 7 – Tue 11, test = Wed 12 – Sun 16.

```
L = live.copy()
L["resid"] = L.log_p95 - mlr.predict(L)
L["bucket"] = L.ts.dt.floor("15min")
p0 = hist.groupby("segment", observed=True).apply(lambda g: g.success_count.sum()
/ g.volume.sum())
ev_vol = hist.groupby(["segment", "hour", "is_weekend"],
observed=True).volume.mean()
L["exp_vol"] = ev_vol.reindex(pd.MultiIndex.from_frame(L[["segment", "hour",
"is_weekend"]])).values
B = (L.groupby(["bucket"] + SEG + ["segment"], observed=True)
       .agg(volume=("volume", "sum"), success=("success_count", "sum"), exp_vol=
("exp_vol", "sum"), resid=("resid", "mean"), p95_ms=("p95_ms", "mean"), n=("ts",
"size"))
       .reset_index())
B = B[B.n == 3]                          # complete scans only (the data-quality gate)
B["p0"] = B.segment.map(p0).astype(float)
B["lat_z"] = B.resid / sigma
B["fail_z"] = (B.p0 - B.success / B.volume) / np.sqrt(B.p0 * (1 - B.p0) /
B.volume)
lv = np.log(B.volume / B.exp_vol); B["vol_drop"], B["vol_jump"] = (-
lv).clip(lower=0), lv.clip(lower=0)
B["ts"] = B.bucket
inc = key[key.kind == "incident"].reset_index(drop=True)
B["y"] = 0
for _, e in inc.iterrows():
    B.loc[in_scope(B, e.scope) & ((B.bucket + pd.Timedelta("15min") > e.start) &
(B.bucket < e.end)).values, "y"] = 1
FEAT = ["lat_z", "fail_z", "vol_drop", "vol_jump"]
split = pd.Timestamp("2025-02-12")
tr, te = B[B.bucket < split], B[B.bucket >= split]
print(f"scans: train {len(tr):,} (positives {tr.y.mean():.2%}) | test {len(te):,}
(positives {te.y.mean():.2%})")
print("→ heavily imbalanced: accuracy would be misleading; we use precision,
recall, F1, ROC-AUC and PR-AUC.")
```

```
scans: train 23,024 (positives 3.60%) | test 23,004 (positives 1.77%)
→ heavily imbalanced: accuracy would be misleading; we use precision, recall, F1,
ROC-AUC and PR-AUC.
```

```
logit = make_pipeline(StandardScaler(),
LogisticRegression(class_weight="balanced", max_iter=1000))
logit.fit(tr[FEAT], tr.y)
lr = logit[-1]
odds = pd.DataFrame({"coef (standardised)": lr.coef_[0], "odds ratio per 1 SD":
np.exp(lr.coef_[0])}, index=FEAT).round(3)
display(odds)
up = odds.index[odds["odds ratio per 1 SD"] > 1].tolist(); down =
odds.index[odds["odds ratio per 1 SD"] < 1].tolist()
say("**Odds ratio** = how much the odds of 'real incident' multiply when the
feature rises by one standard deviation, holding the others fixed.  \n"
    f"* **{', '.join(up)}** raise the odds: slow payments and failures are the
signature of a real incident.  \n"
    f"* **{', '.join(down)}** *lower* the odds. This is a real finding, not a bug:
in the training days the big volume swings came from the "
    "**winter storm** and normal demand, which are *not* incidents. Volume change
on its own says 'demand', and only together with "
    "slowness or failure does it say 'fault'. This is exactly why Lumio treats
system-wide volume swings as advisories.")
# statsmodels version for p-values (inference view)
lm = sm.Logit(tr.y, sm.add_constant((tr[FEAT] - tr[FEAT].mean()) /
tr[FEAT].std())).fit(disp=False)
print(lm.summary2().tables[1][["Coef.", "Std.Err.", "P>|z|"]].round(4))
```

	coef (standardised)	odds ratio per 1 SD
lat_z	1.786	5.965
fail_z	0.948	2.581
vol_drop	-0.379	0.685
vol_jump	-1.647	0.193

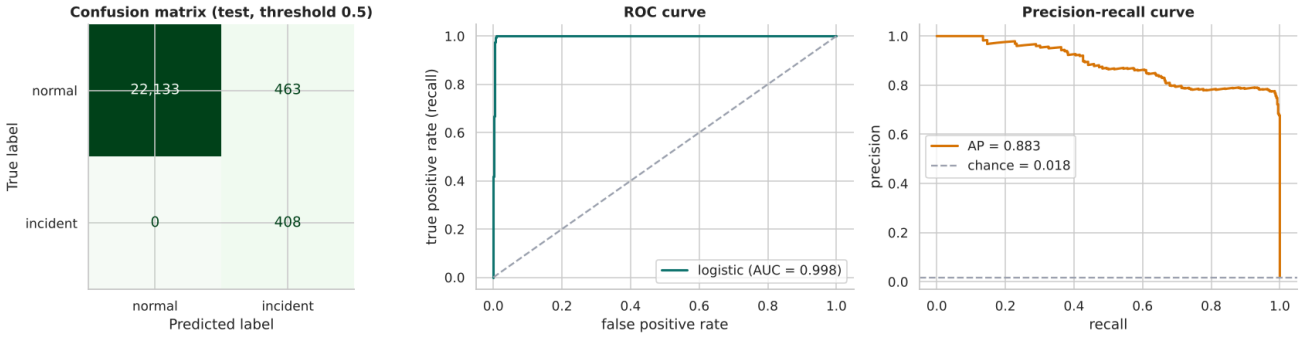
Odds ratio = how much the odds of 'real incident' multiply when the feature rises by one standard deviation, holding the others fixed.

- **lat_z, fail_z** raise the odds: slow payments and failures are the signature of a real incident.
- **vol_drop, vol_jump** *lower* the odds. This is a real finding, not a bug: in the training days the big volume swings came from the **winter storm** and normal demand, which are *not* incidents. Volume change on its own says 'demand', and only together with slowness or failure does it say 'fault'. This is exactly why Lumio treats system-wide volume swings as advisories.

```
           Coef.  Std.Err.  P>|z|
const    -4.7871    0.0877    0.0
lat_z     1.2454    0.0296    0.0
fail_z    0.7957    0.0747    0.0
vol_drop -0.2412    0.0515    0.0
vol_jump -1.2687    0.1561    0.0
```

```
te = te.copy(); te["prob"] = logit.predict_proba(te[FEAT])[:, 1]
te["pred"] = (te.prob >= .5).astype(int)
fig, ax = plt.subplots(1, 3, figsize=(16, 4.2))
ConfusionMatrixDisplay(confusion_matrix(te.y, te.pred), display_labels=["normal",
"incident"]).plot(ax=ax[0], cmap="Greens", colorbar=False, values_format=",")
ax[0].set_title("Confusion matrix (test, threshold 0.5)")
fpr, tpr, _ = roc_curve(te.y, te.prob); auc = roc_auc_score(te.y, te.prob)
ax[1].plot(fpr, tpr, color=TEAL, lw=2, label=f"logistic (AUC = {auc:.3f})");
ax[1].plot([0, 1], [0, 1], ls="--", color=GREY)
ax[1].set(title="ROC curve", xlabel="false positive rate", ylabel="true positive
```

```
rate (recall)"); ax[1].legend()
pr, rc, th = precision_recall_curve(te.y, te.prob); ap =
average_precision_score(te.y, te.prob)
ax[2].plot(rc, pr, color=AMBER, lw=2, label=f"AP = {ap:.3f}");
ax[2].axhline(te.y.mean(), ls="--", color=GREY, label=f"chance =
{te.y.mean():.3f}")
ax[2].set(title="Precision-recall curve", xlabel="recall", ylabel="precision");
ax[2].legend()
plt.tight_layout(); plt.show()
print(classification_report(te.y, te.pred, target_names=["normal", "incident"],
digits=3))
RESULTS["logit_auc"], RESULTS["logit_ap"] = auc, ap
```



	precision	recall	f1-score	support
normal	1.000	0.980	0.990	22596
incident	0.468	1.000	0.638	408
accuracy			0.980	23004
macro avg	0.734	0.990	0.814	23004
weighted avg	0.991	0.980	0.983	23004

8.1 Comparing detectors: fixed threshold vs z-score vs logistic regression

- **Fixed threshold:** p95 > 3,000 ms **or** success < 97 % (what most dashboards do)
- **z-score rule:** lat_z > 3 or fail_z > 3 or |log volume ratio| > 1 (Lumio's statistical core, before its extra guardrails)
- **Logistic regression** at a threshold chosen on the *training* data for F1

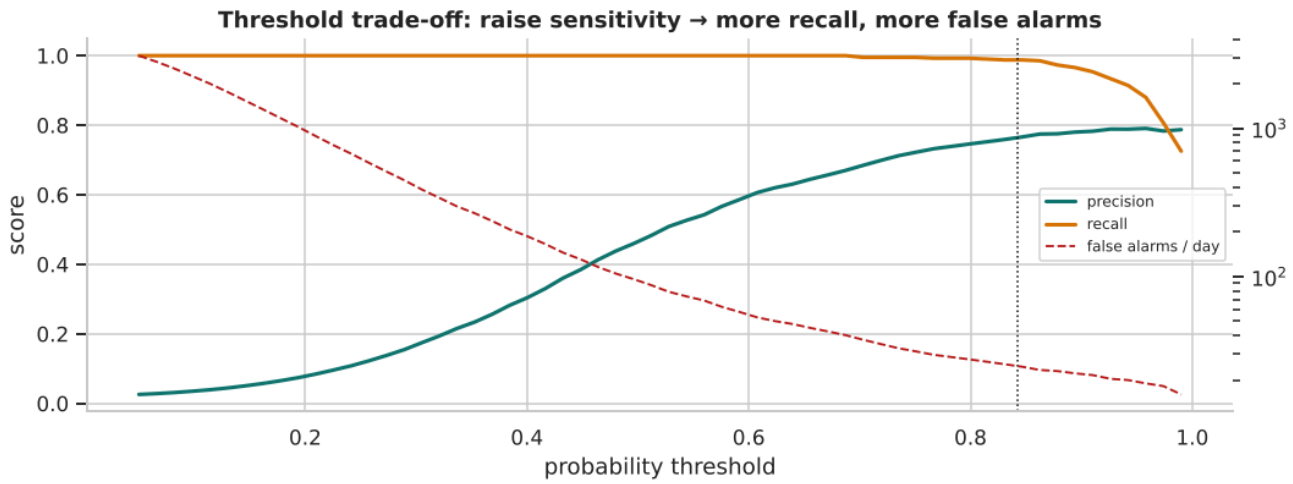
Two levels of evaluation: **scan level** (each segment × 15 min) and **event level** (was each incident caught, and how fast?).

```
pr_t, rc_t, th_t = precision_recall_curve(tr.y, logit.predict_proba(tr[FEAT])[:,
1])
f1_t = 2 * pr_t * rc_t / (pr_t + rc_t + 1e-9); best_th =
th_t[np.argmax(f1_t[:-1])]
rules = {
    "Fixed threshold": ((te.p95_ms > 3000) | (te.success / te.volume <
.97)).astype(int),
    "z-score rule":    ((te.lat_z > 3) | (te.fail_z > 3) | (te.vol_drop > 1) |
(te.vol_jump > 1)).astype(int),
    f"Logistic (thr {best_th:.2f})": (te.prob >= best_th).astype(int)}
inc_te = inc[inc.start >= split]
def event_level(flag):
    out = []
```

```
    for _, e in inc_te.iterrows():
        m = in_scope(te, e.scope) & ((te.bucket + pd.Timedelta("15min") > e.start)
& (te.bucket < e.end)).values & (flag.values == 1)
        first = te.bucket[m].min()
        out.append(np.nan if pd.isna(first) else max(0, (first +
pd.Timedelta("15min") - e.start).total_seconds() / 60))
    return np.array(out)
days = te.bucket.dt.normalize().nunique()
rows = []
for name, fl in rules.items():
    ttd = event_level(fl)
    rows.append({"detector": name, "precision": precision_score(te.y, fl,
zero_division=0), "recall": recall_score(te.y, fl), "F1": f1_score(te.y, fl),
                 "incidents caught": f"{np.isfinite(ttd).sum()}/{len(ttd)}",
"median minutes to detect": np.nanmedian(ttd) if np.isfinite(ttd).any() else
np.nan,
                 "false-alarm scans / day": ((fl == 1) & (te.y == 0)).sum() /
days})
cmp = pd.DataFrame(rows).set_index("detector").round(3); display(cmp);
RESULTS["detectors"] = cmp
```

detector	precision	recall	F1	incidents caught	median minutes to detect	false-alarm scans / day
Fixed threshold	0.113	1.000	0.202	5/5	15.0	643.2
z-score rule	0.628	0.975	0.764	5/5	15.0	47.2
Logistic (thr 0.84)	0.765	0.988	0.862	5/5	15.0	24.8

```
# Threshold trade-off: the sensitivity slider in the app
ths = np.linspace(.05, .99, 60)
curve = pd.DataFrame([{"threshold": t, "precision": precision_score(te.y, te.prob
>= t, zero_division=0),
                       "recall": recall_score(te.y, te.prob >= t), "false
alarms/day": ((te.prob >= t) & (te.y == 0)).sum() / days} for t in ths])
fig, ax = plt.subplots(figsize=(11, 3.6)); ax2 = ax.twinx()
ax.plot(curve.threshold, curve.precision, color=TEAL, lw=2, label="precision");
ax.plot(curve.threshold, curve.recall, color=AMBER, lw=2, label="recall")
ax2.plot(curve.threshold, curve["false alarms/day"], color=RED, lw=1.2, ls="--",
label="false alarms / day"); ax2.set_yscale("log"); ax2.grid(False)
ax.axvline(best_th, color=INK, ls=":", lw=1); ax.set(title="Threshold trade-off:
raise sensitivity → more recall, more false alarms", xlabel="probability
threshold", ylabel="score")
h1, l1 = ax.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels();
ax.legend(h1 + h2, l1 + l2, loc="center right", fontsize=8); plt.show()
say("There is no free lunch: every threshold trades missed incidents against false
alarms. The right point is a **business decision** "
    "(cost of a missed incident vs the cost of waking someone up). Section 11 puts
money on that trade-off.")
```



There is no free lunch: every threshold trades missed incidents against false alarms. The right point is a **business decision** (cost of a missed incident vs the cost of waking someone up). Section 11 puts money on that trade-off.

Why does the production system add more than a classifier? A per-scan model still produces scattered false positives across 48 segments × 96 scans a day. Lumio adds (1) a **data-quality gate**, (2) **grouping** of segments into one story, (3) **confirmation over 2 scans**, (4) the **change calendar** for planned maintenance and (5) **breadth** checks for demand events. Over the full 10 days the live app scores **19/19 planted events with zero false alarms** (see its *Proof* tab), against ~4,900 pages from a fixed rule. Section 11.4 simulates why confirmation is so powerful.

---

9. Machine learning & explainability

Lumio's production model is a **gradient-boosted tree** (HistGradientBoosting). Is it better than linear regression, and can we explain it?

Time-series cross-validation: each fold trains on the past and tests on the *next* block, never on the future. This avoids data leakage.

```
F_ML = ["bank", "channel", "platform", "region", "hour", "dow", "is_weekend",
"log_volume", "avg_amount"]
def to_X(df):
    X = df[F_ML].copy()
    for c in SEG: X[c] = X[c].cat.codes
    for c in ["hour", "dow", "is_weekend"]: X[c] = X[c].astype(float)
    return X
H = hist.sort_values("ts"); Xh, yh = to_X(H), H.log_p95
Xlin = pd.get_dummies(H[F_ML].assign(hour=H.hour.astype(str),
dow=H.dow.astype(str)), drop_first=True).astype(float)
cat_mask = [c in SEG for c in F_ML]
hgb = HistGradientBoostingRegressor(max_iter=300, learning_rate=.08,
categorical_features=cat_mask, random_state=SEED)
tscv = TimeSeriesSplit(n_splits=5)
cv_rows = []
for name, model, X in [("Linear regression", LinearRegression(), Xlin), ("Gradient
boosting", hgb, Xh)]:
```

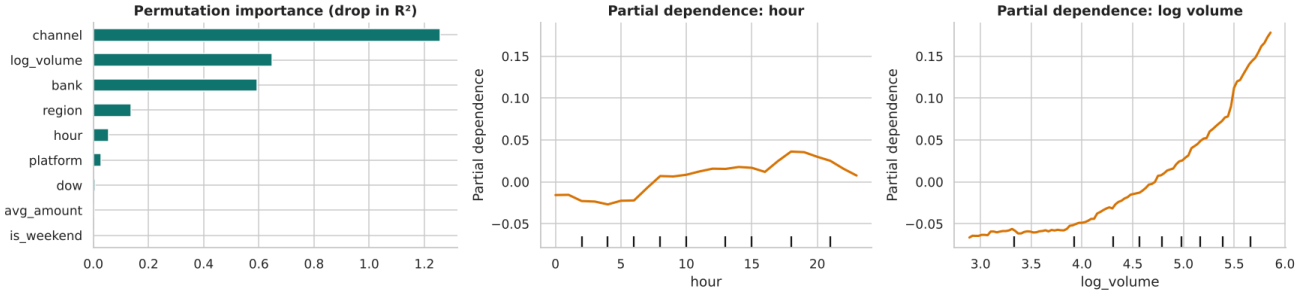
```
    s = cross_validate(model, X, yh, cv=tscv, scoring=["r2",
"neg_root_mean_squared_error"])
    cv_rows.append({"model": name, "CV R² mean": s["test_r2"].mean(), "CV R² sd":
s["test_r2"].std(), "CV RMSE (log)": -
s["test_neg_root_mean_squared_error"].mean()})
cvt = pd.DataFrame(cv_rows).set_index("model").round(4); display(cvt)

hgb.fit(Xh, yh)
T = live[~live.any_event]
p_h = np.exp(hgb.predict(to_X(T)))
RESULTS["hgb_r2_test"] = r2_score(T.p95_ms, p_h); RESULTS["hgb_mae"] =
mean_absolute_error(T.p95_ms, p_h)
say(f"Live-period test (normal rows): gradient boosting R² = **
{RESULTS['hgb_r2_test']:.3f}**, MAE = **{RESULTS['hgb_mae']:.0f} ms** "
    f"vs linear regression R² = {RESULTS['mlr_r2_test']:.3f}, MAE =
{RESULTS['mlr_mae']:.0f} ms. "
    "Trees capture interactions (e.g. *Card at 6 pm on a weekday*) that a linear
model needs to be told about explicitly; the price is interpretability, which we
recover below.")
```

model	CV R² mean	CV R² sd	CV RMSE (log)
Linear regression	0.6330	0.0105	0.1085
Gradient boosting	0.6363	0.0124	0.1080

Live-period test (normal rows): gradient boosting R² = **0.616**, MAE = **190 ms** vs linear regression R² = 0.606, MAE = 192 ms. Trees capture interactions (e.g. *Card at 6 pm on a weekday*) that a linear model needs to be told about explicitly; the price is interpretability, which we recover below.

```
smp = T.sample(15000, random_state=SEED)
pi = permutation_importance(hgb, to_X(smp), smp.log_p95, n_repeats=5,
random_state=SEED, scoring="r2")
imp = pd.Series(pi.importances_mean, index=F_ML).sort_values()
fig, ax = plt.subplots(1, 3, figsize=(16, 3.8), gridspec_kw={"width_ratios": [1.1,
1, 1]})
imp.plot(kind="barh", color=TEAL, ax=ax[0]); ax[0].set(title="Permutation
importance (drop in R²)", xlabel="")
PartialDependenceDisplay.from_estimator(hgb, to_X(smp), ["hour", "log_volume"],
ax=ax[1:], line_kw={"color": AMBER, "lw": 2})
ax[1].set_title("Partial dependence: hour"); ax[2].set_title("Partial dependence:
log volume")
plt.tight_layout(); plt.show()
say(f"Most important driver: **{imp.index[-1]}**, then **{imp.index[-2]}** and **
{imp.index[-3]}**. "
    "Partial-dependence plots show *how* the expected payment time moves with each
feature, which is how we explain the model to non-technical stakeholders.")
```



Most important driver: **channel**, then **log_volume** and **bank**. Partial-dependence plots show *how* the expected payment time moves with each feature, which is how we explain the model to non-technical stakeholders.

---

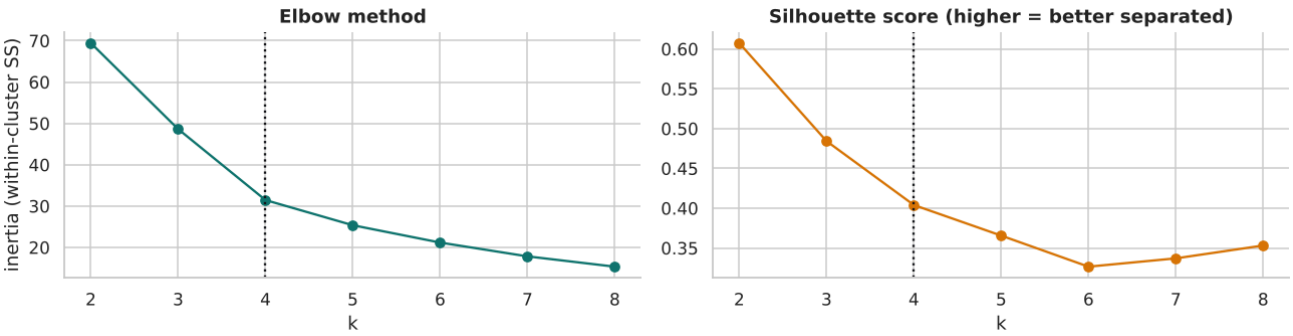
10. Clustering & segmentation (K-means + PCA)

Can we group the 48 segments by **operational behaviour**, without telling the algorithm their labels? Each segment is described by five features from the history:

Feature	Meaning
`p95`	Average payment time
`variability`	Coefficient of variation of p95 (how jumpy it is)
`success`	Overall success rate
`amount`	Average payment value
`log_size`	Log of average volume per 5 minutes

(We first tried the 24-hour demand shape as features too, but every segment follows the same real NAB demand rhythm, so those features carried no information and only added noise. Feature selection is part of the analysis.)

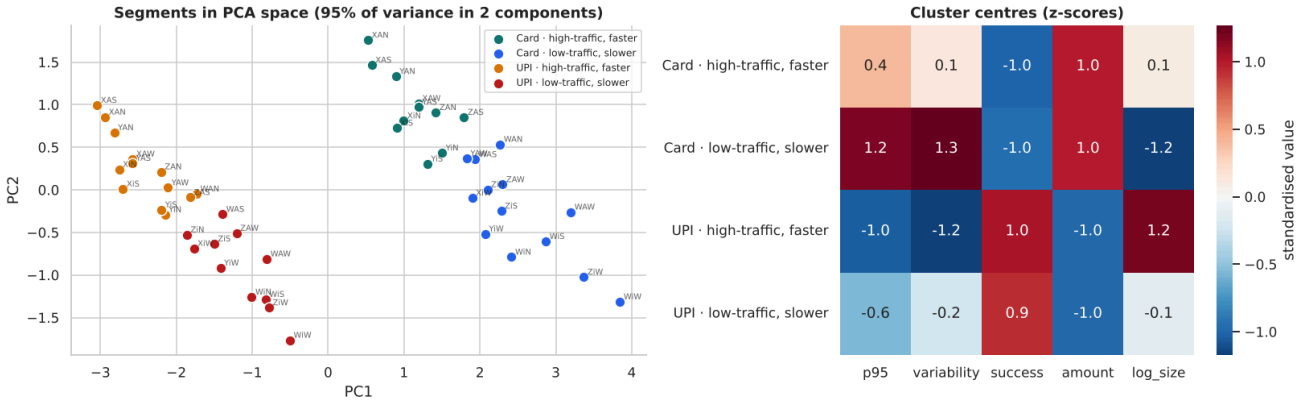
```
g = hist.groupby("segment", observed=True)
Xc = pd.DataFrame({"p95": g.p95_ms.mean(), "variability": g.p95_ms.std() /
g.p95_ms.mean(),
                   "success": g.success_count.sum() / g.volume.sum(), "amount":
g.avg_amount.mean(), "log_size": np.log(g.volume.mean())})
Z = StandardScaler().fit_transform(Xc)          # K-means uses distances, so
features must be on the same scale
ks = range(2, 9); inertia, sil = [], []
for k in ks:
    km = KMeans(k, n_init=20, random_state=SEED).fit(Z);
inertia.append(km.inertia_); sil.append(silhouette_score(Z, km.labels_))
K_FINAL = 4
fig, ax = plt.subplots(1, 2, figsize=(12, 3.3))
ax[0].plot(ks, inertia, "o-", color=TEAL); ax[0].axvline(K_FINAL, ls=":",
color=INK); ax[0].set(title="Elbow method", xlabel="k", ylabel="inertia (within-
cluster SS)")
ax[1].plot(ks, sil, "o-", color=AMBER); ax[1].axvline(K_FINAL, ls=":", color=INK);
ax[1].set(title="Silhouette score (higher = better separated)", xlabel="k")
plt.tight_layout(); plt.show()
k_sil = list(ks)[int(np.argmax(sil))]
say(f"The silhouette score peaks at **k = {k_sil}** ({max(sil):.2f}): that split
is simply **UPI vs Card**, which we already knew. "
    f"We choose **k = {K_FINAL}** (silhouette {sil[K_FINAL-2]:.2f}, still
reasonable structure; the elbow flattens after it) because it reveals something
*new* inside each channel. "
    "Choosing k is a **business judgement informed by metrics**, not a number the
algorithm hands you.")
```



The silhouette score peaks at **k = 2** (0.61): that split is simply **UPI vs Card**, which we already knew. We choose **k = 4** (silhouette 0.40, still reasonable structure; the elbow flattens after it) because it reveals something *new* inside each channel. Choosing k is a **business judgement informed by metrics**, not a number the algorithm hands you.

```
km = KMeans(K_FINAL, n_init=20, random_state=SEED).fit(Z); Xc["cluster"] =
km.labels_
meta = pd.Series(Xc.index.astype(str)).str.split(" · ", expand=True); meta.columns
= SEG; meta.index = Xc.index
Xc["channel"] = meta.channel
prof = Xc.groupby("cluster").agg(segments=("p95", "size"), channel=("channel",
lambda s: s.value_counts().idxmax()),
                                 p95=("p95", "mean"), variability=("variability",
"mean"), success=("success", "mean"),
                                 amount=("amount", "mean"), payments_per_5min=
("log_size", lambda s: np.exp(s).mean()))
ch_mean = prof.groupby("channel").payments_per_5min.transform("mean")
prof["name"] = prof.channel + np.where(prof.payments_per_5min >= ch_mean, " ·
high-traffic, faster", " · low-traffic, slower")
Xc["name"] = Xc.cluster.map(prof.name)
pca = PCA(2).fit(Z); P = pca.transform(Z)
pal = dict(zip(sorted(prof.name), [TEAL, BLUE, AMBER, RED]))
fig, ax = plt.subplots(1, 2, figsize=(15, 4.8), gridspec_kw={"width_ratios": [1.3,
1]})
for n in sorted(prof.name):
    m = Xc.name.values == n; ax[0].scatter(P[m, 0], P[m, 1], s=70, color=pal[n],
label=n, edgecolor="white")
for i, s_ in enumerate(Xc.index):
    ax[0].annotate(meta.loc[s_, "bank"][-1] + meta.loc[s_, "platform"][0] +
meta.loc[s_, "region"][0], P[i], fontsize=6.5, alpha=.7, xytext=(3, 2),
textcoords="offset points")
ax[0].set(title=f"Segments in PCA space ({pca.explained_variance_ratio_.sum():.0%}
of variance in 2 components)", xlabel="PC1", ylabel="PC2");
ax[0].legend(fontsize=8)
cent = pd.DataFrame(km.cluster_centers_, columns=Xc.columns[:5], index=
[prof.name[c] for c in range(K_FINAL)]).sort_index()
sns.heatmap(cent, annot=True, fmt=".1f", cmap="RdBu_r", center=0, ax=ax[1],
cbar_kws={"label": "standardised value"})
ax[1].set(title="Cluster centres (z-scores)", ylabel=""); plt.tight_layout();
plt.show()
display(prof.set_index("name").round(3))
RESULTS["k_best"] = K_FINAL
say("Point labels: bank letter + **A**ndroid/**i**OS + region initial.  \n"
    "**Interpretation:** inside each channel, K-means separates large, fast
segments from small segments that are *slower and more variable*. "
    "The small tier is where a fixed threshold misfires most (noisy, few
payments), so **Lumio's per-segment, noise-scaled thresholds matter most there**.
")
```

```
    "Operationally, clusters can share learned patterns, and an alert hitting a
whole cluster points to a shared cause.")
```



name	segments	channel	p95	variability	success	amount	payments_per_5min
Card · low-traffic, slower	13	Card	2706.290	0.143	0.981	64.094	71.759
UPI · low-traffic, slower	11	UPI	2186.267	0.126	0.983	38.054	119.832
Card · high-traffic, faster	11	Card	2473.636	0.130	0.981	64.077	135.522
UPI · high-traffic, faster	13	UPI	2042.644	0.115	0.983	38.051	237.501

Point labels: bank letter + **A**ndroid/**i**OS + region initial.
Interpretation: inside each channel, K-means separates large, fast segments from small segments that are *slower and more variable*. The small tier is where a fixed threshold misfires most (noisy, few payments), so **Lumio's per-segment, noise-scaled thresholds matter most there**. Operationally, clusters can share learned patterns, and an alert hitting a whole cluster points to a shared cause.

---

11. Monte Carlo simulation

Monte Carlo answers questions that have **uncertain inputs** by simulating thousands of possible futures and looking at the *distribution* of outcomes, not just one average.

11.1 Step 1: measure what an incident costs, from the data

For each real incident in the live period:

- **Excess failed payments** = actual failures − (expected failure rate × volume)
- **Vanished payments**, measured with **difference-in-differences**: how far the affected segments' volume fell below expectation *beyond* how far the unaffected segments (the **control group**) fell in the same window. Without a control group, a quiet Sunday night or the storm would be wrongly counted as lost payments.
- **Value at risk** = (excess failed + vanished) × average payment value, then divided by the incident's minutes.

```
rows = []
for _, e in inc.iterrows():
    m = in_scope(live, e.scope) & ((live.ts >= e.start) & (live.ts <
```

```
e.end)).values
    d = live[m]
    exp_fail = (d.volume * (1 - d.segment.map(p0).astype(float))).sum()
    excess_fail = max(0, d.failed.sum() - exp_fail)
    # vanished payments: difference-in-differences against the unaffected segments
(control group) in the same window
    ctrl = live[~m & ((live.ts >= e.start) & (live.ts < e.end)).values]
    expv = lambda x: ev_vol.reindex(pd.MultiIndex.from_frame(x[["segment", "hour",
"is_weekend"]])).values.sum()
    exp_in = expv(d); ratio_in, ratio_ctrl = d.volume.sum() / exp_in,
ctrl.volume.sum() / expv(ctrl)
    lost = max(0, (ratio_ctrl - ratio_in) * exp_in) if ratio_ctrl - ratio_in > .05
else 0   # ignore < 5 pts (noise)
    minutes = (e.end - e.start).total_seconds() / 60
    value = (excess_fail + lost) * d.avg_amount.mean()
    rows.append({"incident": e["name"][:48], "minutes": minutes, "excess failed":
round(excess_fail), "vanished payments": round(lost),
                 "value at risk $": round(value), "$ per minute": round(value /
minutes, 1)})
impact = pd.DataFrame(rows); display(impact)
RATE = impact["$ per minute"].values
say(f"Median **${np.median(RATE):,.0f} per minute** of incident, max
**${RATE.max():,.0f}**. Latency-only incidents show $0 here because slow-but-
successful payments are not counted as lost. "
    "That is a deliberately **conservative** choice (in reality some customers
abandon a slow payment).")
```

	incident	minutes	excess failed	vanished payments	value at risk $	$ per minute
0	Card network partial outage	40.0	1109	0	71282	1782.0
1	Bank X Android v5.2 slows UPI payments	255.0	1689	0	64485	252.9
2	Card-testing bot attack on Bank Y cards	60.0	21083	0	499803	8330.1
3	UPI switch memory leak (slow drift)	360.0	0	0	0	0.0
4	Bank Z iOS v3.9 login bug — payments vanish	105.0	0	5099	261210	2487.7
5	Regional internet outage: West	60.0	1139	0	57633	960.6
6	Fraud-check database saturates (real NAB level s	90.0	6	0	391	4.3
7	High-value card payments time out at Bank Y	150.0	1525	0	97579	650.5
8	UPI switch partial outage	45.0	3411	0	129781	2884.0
9	Bank W core banking failure (real NAB system fai	70.0	577	0	29415	420.2
10	Bank X core banking saturates during the payday	90.0	1538	0	78901	876.7

Median **$877 per minute** of incident, max **$8,330**. Latency-only incidents show $0 here because slow-but-successful payments are not counted as lost. That is a deliberately **conservative** choice (in reality some customers abandon a slow payment).

11.2 Step 2: the simulation model

Loss per incident = loss rate ($/min) × (time to detect + time to fix)

Input	Distribution	Source
Incidents per year	Poisson(λ = 48, i.e. 4 per month)	Assumption (the demo is deliberately denser)
Loss rate $/min	Bootstrap from the measured incidents above × LogNormal(0, 0.3) noise	**Data** (Section 11.1)

Input	Distribution	Source
Time to detect, **without** Lumio	LogNormal, median 90 min, σ = 0.5	Assumption: customer complaints / manual dashboard checks
Time to detect, **with** Lumio	LogNormal, median 30 min, σ = 0.3	Data: 2 confirmed 15-min scans (live app)
Lumio misses an incident	Bernoulli(5 %) → falls back to "without"	Assumption (conservative: 0 misses in the demo)
Time to fix after detection	LogNormal, median 45 min, σ = 0.6 (same in both worlds)	Assumption (conservative: ignores Lumio's root-cause help)
Annual cost of running Lumio	$150,000	Assumption

10,000 simulated years.

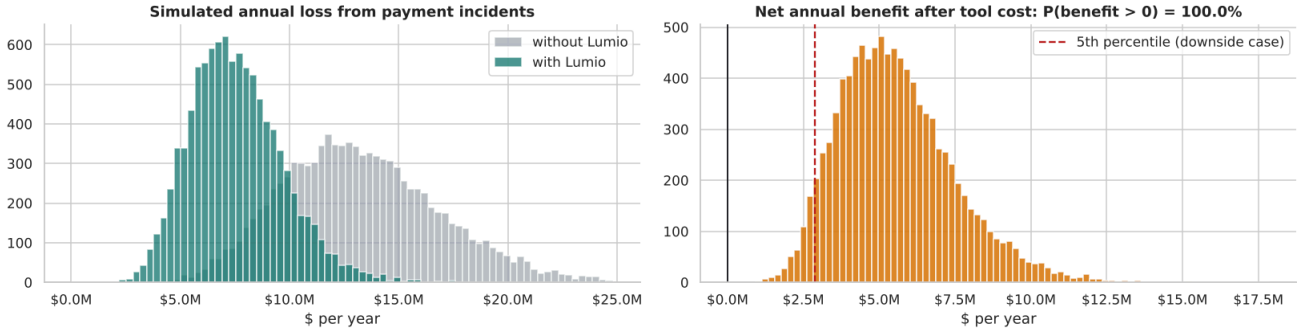
```
BASE = dict(lam=48, ttd0=90, ttd1=30, fix=45, miss=.05, rate_mult=1.0,
cost=150_000)
def simulate(n=10_000, seed=SEED, **kw):
    p = {**BASE, **kw}; g = np.random.default_rng(seed)
    k = g.poisson(p["lam"], n); sim = np.repeat(np.arange(n), k); N = k.sum()
    rate = g.choice(RATE, N) * g.lognormal(0, .3, N) * p["rate_mult"]
    fix = g.lognormal(np.log(p["fix"]), .6, N)
    t0 = g.lognormal(np.log(p["ttd0"]), .5, N)
    t1 = np.where(g.random(N) < p["miss"], t0, g.lognormal(np.log(p["ttd1"]), .3,
N))
    without = np.bincount(sim, rate * (t0 + fix), minlength=n); with_ =
np.bincount(sim, rate * (t1 + fix), minlength=n)
    return pd.DataFrame({"loss_without": without, "loss_with": with_, "saving":
without - with_, "net_benefit": without - with_ - p["cost"]})
t0 = time.time(); mc = simulate(); print(f"10,000 years simulated in {time.time()-
t0:.2f}s")
pct = mc.quantile([.05, .5, .95]).T; pct["mean"] = mc.mean(); pct.columns = ["P5",
"P50 (median)", "P95", "mean"]
display((pct / 1e6).round(2).rename(lambda s: s + " ($M)"))
p_pos = (mc.net_benefit > 0).mean(); RESULTS["mc"] = (mc.saving.median(), p_pos,
mc.net_benefit.quantile(.05))
```

```
10,000 years simulated in 0.07s
```

	P5	P50 (median)	P95	mean
loss_without ($M)	7.87	12.98	19.78	13.30
loss_with ($M)	4.47	7.38	11.32	7.58
saving ($M)	3.04	5.50	9.23	5.72
net_benefit ($M)	2.89	5.35	9.08	5.57

```
fig, ax = plt.subplots(1, 2, figsize=(15, 4))
bins = np.linspace(0, mc.loss_without.quantile(.995), 80)
ax[0].hist(mc.loss_without, bins=bins, color=GREY, alpha=.7, label="without
Lumio"); ax[0].hist(mc.loss_with, bins=bins, color=TEAL, alpha=.75, label="with
Lumio")
ax[0].set(title="Simulated annual loss from payment incidents", xlabel="$ per
year"); ax[0].legend()
ax[0].xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"${x/1e6:.1f}M"))
ax[1].hist(mc.net_benefit, bins=80, color=AMBER, alpha=.85); ax[1].axvline(0,
color=INK, lw=1.2)
ax[1].axvline(mc.net_benefit.quantile(.05), color=RED, ls="--", label="5th
percentile (downside case)")
ax[1].set(title=f"Net annual benefit after tool cost: P(benefit > 0) =
{p_pos:.1%}", xlabel="$ per year"); ax[1].legend()
ax[1].xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"${x/1e6:.1f}M"))
```

```
plt.tight_layout(); plt.show()
say(f"Median annual saving **${mc.saving.median()/1e6:.2f}M**; **{p_pos:.1%}** of
simulated years beat the tool's cost; "
    f"in a bad (5th-percentile) year the net benefit is
**${mc.net_benefit.quantile(.05)/1e6:.2f}M**. "
    f"A single-number business case would hide this spread; Monte Carlo shows the
**risk** as well as the return.")
```



Median annual saving **$5.50M**; **100.0%** of simulated years beat the tool's cost; in a bad (5th-percentile) year the net benefit is **$2.89M**. A single-number business case would hide this spread; Monte Carlo shows the **risk** as well as the return.

For whom is it worth it? The simulated network moves ~2 million payments a day. A smaller bank has proportionally smaller losses but the same tool cost. Scaling the loss rate shows where the business case breaks even.

```
scen = []
for share in [1.0, .25, .10, .05, .02, .01]:
    r = simulate(10_000, rate_mult=share)
    scen.append({"bank size (share of this network)": f"{share:.0%}", "payments /
day ≈": f"{2_000_000*share:,.0f}",
                 "median saving $": r.saving.median(), "median net benefit $":
r.net_benefit.median(), "P(net benefit > 0)": (r.net_benefit > 0).mean()})
scen = pd.DataFrame(scen).set_index("bank size (share of this network)")
display(scen.style.format({"median saving $": "${:,.0f}", "median net benefit $":
"${:,.0f}", "P(net benefit > 0)": "{:.1%}"}))
be = scen[scen["P(net benefit > 0)"] >= .5].index[-1]
RESULTS["mc_breakeven"] = be
say(f"For a network this size the case is overwhelming; it stays positive in most
simulated years down to roughly **{be}** of this size. "
    "Below that, a shared or cheaper deployment is needed. A probability statement
like this is far more useful to a decision-maker than one ROI number.")
```

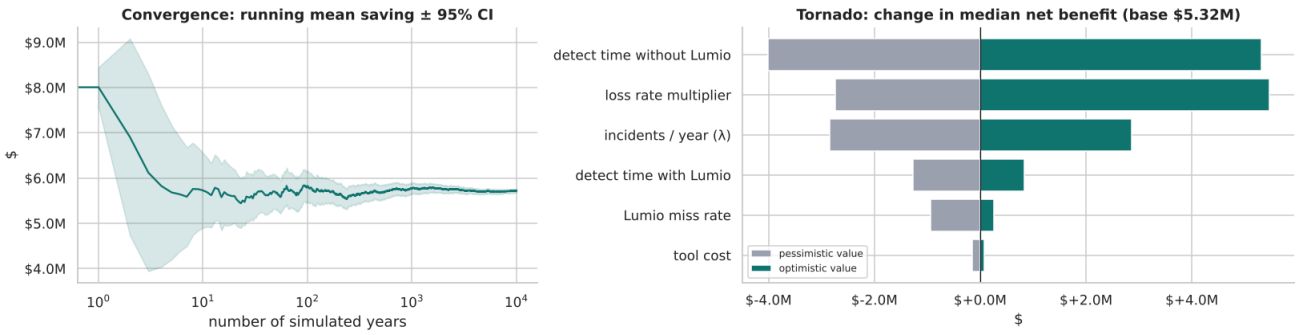
bank size (share of this network)	payments / day ≈	median saving $	median net benefit $	P(net benefit > 0)
100%	2,000,000	$5,502,422	$5,352,422	100.0%
25%	500,000	$1,375,606	$1,225,606	100.0%
10%	200,000	$550,242	$400,242	99.9%
5%	100,000	$275,121	$125,121	95.4%
2%	40,000	$110,048	$-39,952	16.5%
1%	20,000	$55,024	$-94,976	0.1%

For a network this size the case is overwhelming; it stays positive in most simulated years down to roughly **5%** of this size. Below that, a shared or cheaper deployment is needed. A probability statement like this is far more useful to a decision-maker than one ROI number.

11.3 Convergence and sensitivity: how much should we trust the simulation?

```
run = mc.saving.expanding().mean(); se = mc.saving.expanding().std() /
np.sqrt(np.arange(1, len(mc) + 1))
fig, ax = plt.subplots(1, 2, figsize=(15, 4), gridspec_kw={"width_ratios": [1,
1.2]})
ax[0].plot(run.values, color=TEAL); ax[0].fill_between(range(len(run)), run - 1.96
* se, run + 1.96 * se, color=TEAL, alpha=.15)
ax[0].set(xscale="log", title="Convergence: running mean saving ± 95% CI",
xlabel="number of simulated years", ylabel="$")
ax[0].yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"${x/1e6:.1f}M"))

ranges = {"incidents / year (λ)": ("lam", 24, 72), "detect time without Lumio":
("ttd0", 45, 150), "detect time with Lumio": ("ttd1", 45, 20),
          "loss rate multiplier": ("rate_mult", .5, 2.0), "Lumio miss rate":
("miss", .2, 0.0), "tool cost": ("cost", 300_000, 75_000)}
base = simulate(4000).net_benefit.median()
tor = pd.DataFrame([{"input": k, "low": simulate(4000, **{v[0]:
v[1]}).net_benefit.median() - base,
                     "high": simulate(4000, **{v[0]: v[2]}).net_benefit.median() -
base} for k, v in ranges.items()])
tor["span"] = (tor.high - tor.low).abs(); tor = tor.sort_values("span")
ax[1].barh(tor.input, tor.low, color=GREY, label="pessimistic value");
ax[1].barh(tor.input, tor.high, color=TEAL, label="optimistic value")
ax[1].axvline(0, color=INK, lw=.8); ax[1].set(title=f"Tornado: change in median
net benefit (base ${base/1e6:.2f}M)", xlabel="$")
ax[1].xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _:
f"${x/1e6:+.1f}M")); ax[1].legend(fontsize=8)
plt.tight_layout(); plt.show()
say(f"The estimate stabilises after a few thousand runs (the CI band narrows as 1/
√n). The result is most sensitive to **{tor.input.iloc[-1]}** and **
{tor.input.iloc[-2]}**: "
    "those are the assumptions to validate with a real bank before a production
business case. "
    "*Time to fix* is not in the chart because it is the same with or without
Lumio, so it cancels out of the saving: a reminder to check which inputs actually
drive the answer.")
```



The estimate stabilises after a few thousand runs (the CI band narrows as 1/√n). The result is most sensitive to **detect time without Lumio** and **loss rate multiplier**: those are the assumptions to validate with a real bank before a production business case. *Time to fix* is not in the chart because it is the same with or without Lumio, so it cancels out of the saving: a reminder to check which inputs actually drive the answer.

11.4 Monte Carlo for alert design: why one scan is never enough

Normal noise only, **no incidents at all**. 48 segments × 3 signals × 96 scans a day = 13,824 checks a day. Noise is autocorrelated (AR(1), ρ = 0.5). How many false pages per day does each rule produce?

```
g = np.random.default_rng(SEED); DAYS, SCANS, CH, rho = 200, 96, 48 * 3, .5
eps = g.standard_normal((DAYS * SCANS, CH)); z = np.empty_like(eps); z[0] = eps[0]
for t in range(1, len(z)): z[t] = rho * z[t - 1] + np.sqrt(1 - rho**2) * eps[t]
def pages_per_day(trig): return trig.reshape(DAYS, SCANS, CH).sum(axis=(1, 2))
rules = {"z > 2, single scan": z > 2, "z > 3, single scan": z > 3,
         "z > 3, 2 scans in a row": (z[1:] > 3) & (z[:-1] > 3), "z > 2.5, 2 scans
in a row": (z[1:] > 2.5) & (z[:-1] > 2.5)}
fa = []
for name, trig in rules.items():
    if trig.shape[0] < DAYS * SCANS: trig = np.vstack([np.zeros((1, CH), bool),
trig])
    ppd = pages_per_day(trig)
    fa.append({"rule": name, "false pages / day (mean)": ppd.mean(), "days with
zero false pages": (ppd == 0).mean(),
               "simulated P(one check fires)": trig.mean(), "theory P(one check
fires)": np.nan})
fa = pd.DataFrame(fa).set_index("rule")
fa.loc["z > 2, single scan", "theory P(one check fires)"] = 1 - stats.norm.cdf(2)
fa.loc["z > 3, single scan", "theory P(one check fires)"] = 1 - stats.norm.cdf(3)
display(fa.round(4)); RESULTS["mc_alert"] = fa
say(f"Even a strict z > 3 fires **{fa.iloc[1,0]:.0f} false pages a day** across
13,824 checks: the **multiple-comparisons problem**. "
    f"Requiring confirmation on 2 consecutive scans cuts that to **
{fa.iloc[2,0]:.1f}**. Adding grouping, minimum volume and the data-quality gate is
how the live app reaches zero. "
    "The simulation also checks the theory: the simulated single-scan rates match
the normal-distribution tail probabilities.")
```

rule	false pages / day (mean)	days with zero false pages	simulated P(one check fires)	theory P(one check fires)
z > 2, single scan	316.545	0.00	0.0229	0.0228
z > 3, single scan	19.120	0.00	0.0014	0.0013
z > 3, 2 scans in a row	1.155	0.36	0.0001	NaN
z > 2.5, 2 scans in a row	9.315	0.00	0.0007	NaN

Even a strict z > 3 fires **19 false pages a day** across 13,824 checks: the **multiple-comparisons problem**. Requiring confirmation on 2 consecutive scans cuts that to **1.2**. Adding grouping, minimum volume and the data-quality gate is how the live app reaches zero. The simulation also checks the theory: the simulated single-scan rates match the normal-distribution tail probabilities.

---

12. Ethics, privacy & governance

Principle	How this project applies it
Data minimisation & privacy	Only aggregated telemetry (counts, rates, timings per segment per 5 min). **No customer names, card numbers or account data** ever enter the analysis. In production, segment data would be pseudonymised

Principle	How this project applies it
	and access-controlled (PIPEDA in Canada, GDPR, PCI-DSS for card data).
Fairness	Thresholds are learned **per segment**, so a smaller bank or quieter region is not ignored, or over-alerted, just because it is small (Sections 5.5, 8).
Transparency / explainability	Every alert states the evidence (charts, the test used, the p-value). Gradient-boosting results are explained with importance and partial dependence (Section 9).
Human in the loop	Lumio recommends; people decide. Actions such as *pause rollout* or *block pattern* need a click from the accountable team, and every action is logged.
AI accountability	When the AI agent proposes a cause, the claim must pass a statistical test (Fisher / Mann-Whitney) before it is shown. No test, no claim: it says "cause unknown" instead of guessing.
Honest evaluation	The answer key is never used to tune detection rules; train/test splits are by time; assumptions in the business case are labelled and stress-tested.
Data provenance & licensing	NAB data used under its MIT licence and cited; simulated components are clearly declared.

---

13. Results, limitations & recommendations

```
f = RESULTS["forecast"]; d = RESULTS["detectors"]; zrow = d.loc["z-score rule"];
frow = d.loc["Fixed threshold"]
say(f"""
### Key results
| Question | Answer from the data |
|---|---|
| Q1 · What is normal? | Strong daily + weekly seasonality (strength
{RESULTS['seasonal_strength'][24]:.2f} / {RESULTS['seasonal_strength'][168]:.2f});
busiest hour ≈ **{RESULTS['peak_trough_ratio']:.0f}×** the quietest |
| Q2 · Do groups differ? | Bank matters (η² = {RESULTS['anova_bank'][1]:.1%},
large); Android vs iOS is a small effect (d = {RESULTS['t_android_ios'][2]:.2f});
channel vs failure is significant but negligible (Cramér's V =
{RESULTS['chi_channel'][1]:.4f}) |
| Q3 · What drives payment time? | UPI switch alone explains R² = {RESULTS['slr']
[1]:.2f} of UPI payment time; multiple regression R² =
{RESULTS['mlr_r2_train']:.2f} (train) / {RESULTS['mlr_r2_test']:.2f} (unseen live
data); gradient boosting R² = {RESULTS['hgb_r2_test']:.2f} |
| Q3b · Root cause | App v5.2 is **{RESULTS['release_ratio']:.1f}× slower** than
v5.1 (Mann-Whitney, p < 0.001) |
| Q4 · Forecasting | Best model **{f['MAPE %'].idxmin()}**, MAPE **{f['MAPE
%'].min():.1f} %** on normal days ({f['skill vs naive %'].max():.0f} % lower error
than seasonal naive); {RESULTS['storm_flag'][0]:.0%} of storm hours fall outside
the 95 % interval vs {RESULTS['storm_flag'][1]:.0%} of other hours |
| Q5 · Detection | Fixed threshold: precision {frow.precision:.2f}, {frow['false-
alarm scans / day']:.0f} false-alarm scans/day · z-score rule: precision
{zrow.precision:.2f}, recall {zrow.recall:.2f}, {zrow['false-alarm scans /
day']:.0f}/day · logistic ROC-AUC **{RESULTS['logit_auc']:.3f}**. All caught
{zrow['incidents caught']} test incidents; the full live app (with grouping +
confirmation) scores 19/19 with zero false alarms |
| Q6 · Segments | K-means finds **{RESULTS['k_best']}** behavioural clusters |
| Q7 · Is it worth it? | Median saving **${RESULTS['mc'][0]/1e6:.2f}M / year** for
this network; P(net benefit > 0) = **{RESULTS['mc'][1]:.1%}**; still positive in
most years down to ~{RESULTS['mc_breakeven']} of this size |
""")
```

Key results

Question	Answer from the data
Q1 · What is normal?	Strong daily + weekly seasonality (strength 0.99 / 0.98); busiest hour ≈ **19×** the quietest
Q2 · Do groups differ?	Bank matters (η^2 = 17.4%, large); Android vs iOS is a small effect (d = 0.34); channel vs failure is significant but negligible (Cramér's V = 0.0072)
Q3 · What drives payment time?	UPI switch alone explains R^2 = 0.85 of UPI payment time; multiple regression R^2 = 0.64 (train) / 0.61 (unseen live data); gradient boosting R^2 = 0.62
Q3b · Root cause	App v5.2 is **3.2× slower** than v5.1 (Mann-Whitney, $p < 0.001$)
Q4 · Forecasting	Best model **SARIMA(1, 0, 1)(0,1,0)$_{168}$**, MAPE **5.9 %** on normal days (15 % lower error than seasonal naive); 82% of storm hours fall outside the 95 % interval vs 6% of other hours
Q5 · Detection	Fixed threshold: precision 0.11, 643 false-alarm scans/day · z-score rule: precision 0.63, recall 0.97, 47/day · logistic ROC-AUC **0.998**. All caught 5/5 test incidents; the full live app (with grouping + confirmation) scores 19/19 with zero false alarms
Q6 · Segments	K-means finds **4** behavioural clusters
Q7 · Is it worth it?	Median saving **$5.50M / year** for this network; P(net benefit > 0) = **100.0%**; still positive in most years down to ~5% of this size

Limitations (be the first to say them)

1. **Simulated payments.** The rhythm and three anomalies are real (NAB), but payment outcomes are simulated. Results show the *method* works; a pilot on a real bank's data is the next step.
2. **One season.** Three weeks of history cannot learn yearly patterns (holidays, tax season). Production needs 12+ months.
3. **Planted incidents are known types.** A truly novel failure may look different; that is why the app includes the AI agent and "cause unknown".
4. **Autocorrelation.** Rows close in time are not independent, so classical p-values are optimistic. We mitigated this with hourly units, time-based splits and time-series CV.
5. **Business-case assumptions.** Incident frequency, detection time without Lumio and tool cost are assumptions, which is why we show their sensitivity.

Recommendations

1. Use **expected-vs-actual** (regression / ML baselines) instead of fixed thresholds for every payment KPI.
2. Require **statistical evidence + 2-scan confirmation** before paging people.
3. Treat **system-wide demand swings** (storms, paydays) as advisories, not incidents.
4. Run a **90-day pilot** with one bank to validate the Monte Carlo assumptions, then re-run Section 11 with real numbers.
5. Re-train baselines **monthly** and monitor for **model drift** (performance falling on recent data).

---

14. Professor's likely questions — and how to answer them

A. Problem & project design

1. Why is this a data analytics problem and not just an IT monitoring problem? Because the failures are *statistical*: every server is "up", but a slice of customers sees slower or failed payments. Spotting them needs baselines, seasonality, hypothesis tests and segmentation, which is analytics work, not uptime checks.

2. Which methodology did you follow? CRISP-DM (Section 0): business understanding → data understanding → preparation → modelling → evaluation → deployment (the live app).

3. Who are the stakeholders and how does your output reach them? Ops/SRE, Release, Fraud, Business Operations, Leadership (Section 0.2). The live app routes each alert by e-mail to the responsible department, with evidence and action buttons.

4. How did you define success before you started? Section 0.4: $R^2 \geq 0.5$ on unseen data, MAPE ≤ 10 %, catching every test incident with fewer false alarms than a fixed rule, and a positive expected net benefit. Section 13 reports against these.

B. Data

5. Is the data real? Partly, and that is deliberate. Volume rhythm, a blizzard, a system failure and a database level-shift come from the **Numenta Anomaly Benchmark** (real, labelled). Payment outcomes are simulated on top so the ground truth is known exactly. Real bank data is confidential *and* unlabelled, so you could not measure precision and recall honestly with it.

6. How did you handle missing data and duplicates? Duplicates removed; gaps **not** filled with zeros, because a zero would fake an outage. IQR outliers kept because they are real peak behaviour (Section 2).

7. Why use p95 instead of the average payment time? Latency is right-skewed; the 95th percentile captures the slow tail customers actually feel and is the industry-standard SLI.

8. Why did you log-transform payment time? To reduce skew, stabilise variance and make coefficients read as % effects (Section 6.3).

C. Statistics

9. Your p-values are all tiny. Does that mean everything matters? No. With ~40 M payments, tiny differences are significant. That is why we report **effect sizes** (Cohen's d, η^2, Cramér's V). Channel vs failure is significant but negligible (Section 5.2).

10. Why Welch's t-test instead of Student's t-test? Levene's test showed unequal variances; Welch does not assume them.

11. Why Mann-Whitney for the release comparison? p95 values are skewed and the samples are small, so a non-parametric test is safer. Fisher's exact test is used for failure counts in a 2×2 table.

12. Isn't running many tests a multiple-comparisons problem? Yes. Section 11.4 simulates it: 13,824 checks a day at $z > 3$ still give dozens of false pages a day. We control it with

confirmation over 2 scans, grouping and minimum-volume rules, the operational equivalent of a family-wise correction. In a purely academic setting, Bonferroni or Benjamini-Hochberg would be the textbook fixes.

D. Regression

13. Interpret one coefficient for me. Use Section 6.3, e.g. "Holding bank, region, hour and load constant, UPI payments are about X % faster than card payments", where X comes from the coefficient table.

14. Did you check the regression assumptions? Yes (Section 6.4): residual plot, Q-Q plot, Breusch-Pagan (heteroscedasticity → robust HC3 errors), Durbin-Watson (autocorrelation in time data), VIF (multicollinearity).

15. What is confounding, and where did you see it? Amount appeared to slow payments, but card payments are both bigger and slower. Adding channel removed the effect (Section 6.2). This is omitted-variable bias.

16. Why is R^2 on training data different from R^2 on the test data? Test data is unseen and from a different period. A small drop is normal; a large drop would mean overfitting. We report both (6.3, 6.5).

17. Linear regression vs gradient boosting: which would you deploy, and why? Gradient boosting fits better (captures interactions), and we explain it with permutation importance and partial dependence. Linear regression stays as the interpretable benchmark and the teaching model (Section 9).

E. Time series

18. What is stationarity and why does it matter? Constant mean and variance over time; ARIMA-type models assume it. ADF and KPSS (opposite nulls) show seasonal differencing is needed (Section 7.2).

19. How did you choose the SARIMA orders? The ACF shows 24 h and 168 h seasonality, so we difference by one week (D = 1, s = 168). The PACF of the differenced series suggests a low order, and we compared candidate ARMA orders by **AIC** and by **validation MAPE** on the last training week, choosing by validation because the goal is forecasting (Section 7.4). A payday regressor was added. It was then tested walk-forward on the 10 live days against seasonal naive and Holt-Winters.

19b. AIC picked a different model from validation. Which is right? AIC measures in-sample fit with a complexity penalty; validation measures accuracy on unseen days. For a forecasting goal, out-of-sample performance wins, and here the validation choice also did better on the test days.

20. What is walk-forward validation and why use it? Each day we refit on everything known up to midnight and forecast the next 24 hours, which is how the forecast would be used in operations. It never lets the model see the future, and it tests the model 10 separate times instead of once.

20b. Why exclude the storm when measuring forecast accuracy? No model should predict a blizzard from payment history. We measure accuracy on normal hours, then show the storm is *detected* because it falls outside the prediction interval (Section 7.5).

21. Why not shuffle the data for cross-validation? Shuffling leaks the future into training. We use time-based splits and TimeSeriesSplit.

F. Classification & evaluation

22. Why not report accuracy? Only about 2–4 % of scans are inside an incident: a model that always says "normal" would be over 96 % accurate and completely useless. We use precision, recall, F1, ROC-AUC and PR-AUC.

23. Precision or recall: which matters more here? Both have a cost: a missed incident loses money; a false alarm wastes people's time and erodes trust (alert fatigue). The threshold curve (Section 8.1) shows the trade-off, and the Monte Carlo puts dollars on it.

24. How do you interpret the logistic-regression odds ratios? The multiplier on the odds of a real incident per 1-SD rise in a feature (Section 8).

25. Your logistic model was trained with labels. Doesn't that use the answer key? Only for this classroom comparison, and only on the training days. The live app's detector never sees labels; it uses the unsupervised expected-vs-actual rules and is scored afterwards.

G. Clustering & Monte Carlo

26. How did you pick k in K-means? Elbow method plus silhouette score (Section 10). The silhouette peaks at k = 2, but that is just UPI vs Card, which we already knew. k = 4 still has reasonable separation and reveals a high-traffic/fast vs low-traffic/slow tier inside each channel, which is actionable. Features were standardised first because K-means uses distances. We also dropped the demand-shape features after finding they carried no information.

27. Why Monte Carlo and not just one ROI number? The inputs are uncertain. Monte Carlo gives a distribution: median saving, probability of a positive return, a bad-year (5th percentile) case, and the bank size where the case breaks even (Section 11).

27b. How did you measure the cost of an incident? From the data: excess failures against the segment's normal failure rate, plus vanished payments measured by **difference-in-differences** against unaffected segments (a control group). Without the control group, a quiet Sunday night or the storm would be miscounted as lost money.

28. How many simulations are enough? Until the running mean stabilises. The convergence plot shows the CI narrowing as 1/√n; 10,000 is well past that.

29. Which assumption matters most? The tornado chart (Section 11.3) ranks them, which tells the bank what to measure first in a pilot.

H. Ethics, limitations & future work

30. Any privacy concerns? Only aggregated, non-personal telemetry is used; production would add pseudonymisation and access controls (PIPEDA / GDPR / PCI-DSS). See Section 12.

31. What are the main limitations? Simulated outcomes, one season of history, known incident types, autocorrelation and business-case assumptions (Section 13).

32. If you had 3 more months, what would you do? Pilot on one bank's real data; 12 months of history for yearly seasonality; drift monitoring and monthly retraining; validate the Monte Carlo inputs; A/B-test alert thresholds with the on-call team.

33. What did you personally learn? *(Answer in your own words.)* For example: that "significant" isn't "important", that the baseline matters more than the threshold, and that the hardest part of analytics is turning a number into an action someone takes.

Data: Numenta Anomaly Benchmark (Lavin & Ahmad, 2015; MIT licence) + simulated payments. Methods: pandas, SQLite, SciPy, statsmodels, scikit-learn.